

**PENERAPAN *MACHINE LEARNING* DAN *DEEP LEARNING*
UNTUK KLASIFIKASI JENIS BUNGA**

**LAPORAN AKHIR
MACHINE LEARNING**



**OLEH:
MUHAMMAD RIZKI RAMADHAN
NIM. 2310817310008**

**DOSEN PEMBIMBING:
ERIKA MAULIDIYA, S.Kom., M.Kom.**

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
BANJARMASIN**

2026

DAFTAR ISI

DAFTAR ISI	i
DAFTAR GAMBAR	iv
DAFTAR TABEL.....	v
DAFTAR LAMPIRAN	Error! Bookmark not defined.
BAB I_PENDAHULUAN	1
1. 1 Latar Belakang	1
1. 2 Rumusan Masalah	2
1. 3 Batasan Masalah	3
1. 4 Tujuan Penelitian	3
1. 5 Manfaat Penelitian	3
BAB II_TINJAUAN PUSTAKA	5
2. 1 Penelitian Terkait	5
2. 1. 1 Model Klasifikasi Bunga.....	5
2. 2 Landasan Teori.....	8
2. 2. 1 Klasifikasi Citra Digital	8
2. 2. 2 Support Vector Machine (SVM)	8
2. 2. 3 Random Forest	9
2. 2. 4 Histogram of Oriented Gradients (HOG).....	10
2. 2. 5 Convolutional Neural network (CNN).....	10
2. 2. 6 VGG16	11
2. 3 MobileNetV2	11
2. 4 Transfer Learning.....	12
BAB III_METODOLOGI PENELITIAN.....	14
3. 1 Dataset.....	14

3. 2 Preprocessing Data.....	16
3. 3 Metode Machine Learning dari UTS	18
3. 3. 1 Support Vector Machine (SVM)	18
3. 3. 2 Random Forest	20
3. 4 Pengembangan Deep Learning pada UAS.....	21
3. 5 Workflow atau Alur Penelitian	24
3. 6 Hasil dan Evaluasi Model	27
3. 6. 1 Model HOG + SVM.....	27
3. 6. 2 Model Random Forest.....	29
3. 6. 3 Model VGG16 Fine-Tuning.....	33
3. 6. 4 Model MobileNetV2	36
3. 6. 5 Model Hybrid VGG16 + SVM	38
3. 6. 6 Model Hybrid MobileNetV2 + Random Forest.....	42
3. 7 Visualisasi Hasil.....	45
3. 7. 1 Model HOG + SVM.....	45
3. 7. 2 Model Random Forest.....	46
3. 7. 3 Model VGG16 Fine-Tuning.....	49
3. 7. 4 MobileNetV2	50
3. 7. 5 Model SVM + VGG16.....	51
3. 7. 6 Model Random Forest + MobileNetV2	53
BAB IV_PEMBAHASAN.....	54
4. 1 Analisis Performa Model Terbaik.....	54
4. 2 Perbandingan Pendekatan Machine Learning dan Deep Learning	54
4. 3 Kendala dalam penelitian.....	55
4. 4 Kelemahan Model	56
4. 5 Peluang Pengembangan ke Depan	56

BAB V_Demo Program	58
5. 1 Spesifikasi Perangkat Lunak dan Pustaka (Libraries)	58
5. 2 Cara Menjalankan Program	58
5. 3 Uji Coba Pengujian	60
BAB VI_Kesimpulan	64
DAFTAR PUSTAKA	65
Lampiran 1 Link Github.....	66
Lampiran 2 Link Dataset.....	67
Screenshot Program dan Potongan Kode Penting.....	68

DAFTAR GAMBAR

Gambar 3. 1 Alur Penelitian	24
Gambar 3. 2 Hasil 5-Fold Cross-Validation.....	30
Gambar 3. 3 Matrik Evaluasi per kelas RF	31
Gambar 3. 4 Confusion Matrix RF	32
Gambar 3. 5 Confusion matrix HOG + SVM.....	45
Gambar 3. 6 perbandingan Cross-Validation	46
Gambar 3. 7 Confusion Matrix Random Forest	47
Gambar 3. 8 Feature Importance Random Forest.....	48
Gambar 3. 9 Confusion matrix VGG16.....	49
Gambar 3. 10 Confusion Matrix MobileNetV2.....	50
Gambar 3. 11 Confusion matrix SVM + VGG16	52
Gambar 3. 12 Confusion Matrix RF + MobileNetV2.....	53

DAFTAR TABEL

Tabel 3. 1 Variasi Konfigurasi HOG pada SVM.....	27
Tabel 3. 2 Percobaan 1 SVM+HOG	27
Tabel 3. 3 Percobaan 2 SVM+HOG	28
Tabel 3. 4 Percobaan 3 SVM+HOG	28
Tabel 3. 5 Perbandingan Percobaan SVM HOG.....	28
Tabel 3. 6 Hasil Cross-Validation	30
Tabel 3. 7 Matrik Evaluasi RF	31
Tabel 3. 8 Classification Report RF	32
Tabel 3. 9 Matriks Evlauasi VGG16.....	34
Tabel 3. 10 Analisis Performa VGG16 per Kelas	35
Tabel 3. 11 Phase 1 Pelatihan MobileNetV2	36
Tabel 3. 12 Phase 2 Pelatihan MobileNetV2	37
Tabel 3. 13 Metrik Evaluasi MobileNetV2.....	38
Tabel 3. 14 Evaluasi Performa MobileNetV2 per kelas	38
Tabel 3. 15 Phase 1 Pelatihan Model VGG16	39
Tabel 3. 16 Phase 2 Pelatihan VGG16.....	39
Tabel 3. 17 Perbandingan kernel.....	40
Tabel 3. 18 Hasil Prediksi Model VGG16 + SVM per Kelas	41
Tabel 3. 19 Phase 1 Pelatihan MobileNetV2	42
Tabel 3. 20 Phase 2 pelatihan MobileNetV2	43
Tabel 3. 21 Metrik Evaluasi MobileNetV2 + RF.....	43
Tabel 3. 22 Hasil Prediksi Model MobileNetV2 + Random Forest per Kelas	44

BAB I

PENDAHULUAN

1.1 Latar Belakang

Indonesia merupakan salah satu negara dengan keanekaragaman hayati tertinggi di dunia salah satunya kekayaan flora. Terdapat ratusan jenis tumbuhan dan bunga yang tersebar di seluruh Indonesia [1]. Beberapa jenis bunga seperti Melati Jakarta (*Jasminum sambac*), Melati Jepang (*Pseuderanthemum reticulatum*), Bintaro (*Cerbera manghas*), dan Tapak Dara (*Catharanthus roseus*) merupakan tanaman hias yang biasanya dijumpai di lingkungan perkotaan. Namun, jenis-jenis bunga yang memiliki kemiripan morfologi seperti keempat jenis bunga tersebut pada beberapa bagian, seperti bentuk mahkota, warna, serta jumlah kelopak bunganya sering kali sulit dibedakan secara visual oleh masyarakat umum. Selain itu, identifikasi manual oleh pakar botani juga membutuhkan waktu yang lama dan tidak praktis untuk melakukan pengenalan pada jenis-jenis bunga yang memiliki kemiripan morfologi dalam skala besar [1].

Perkembangan teknologi kecerdasan buatan, khususnya pada bidang *machine learning* dan *deep learning*, telah membuka peluang besar untuk membangun dan mengembangkan sistem klasifikasi objek berbasis data citra gambar yang otomatis dan akurat. Pendekatan berbasis *machine learning* seperti *Random Forest* telah banyak digunakan pada berbagai tugas klasifikasi citra dengan memanfaatkan ekstraksi fitur manual [2]. Selain itu, kombinasi antara *machine learning* dan *deep learning* seperti CNN dengan *Random Forest* untuk klasifikasi gambar juga dapat meningkatkan akurasi generalisasi dibandingkan jika hanya dengan penggunaan *Random Forest* tunggal berbasis fitur manual [3].

Metode *machine learning* memang telah terbukti dapat melakukan tugas klasifikasi citra dengan akurasi yang tinggi, namun teknologi ini memiliki keterbatasan dalam menangkap representasi fitur yang kompleks dan berhierarki dari data gambar, terutama ketika terdapat berbagai kondisi gambar, seperti variasi pencahayaan, sudut pandang pengambilan gambar, dan latar belakang pada gambar yang beragam [4]. Pendekatan *deep learning* berbasis *Convolutional Neural Network* (CNN) dengan teknik *transfer learning* telah terbukti mampu mengatasi

keterbatasan tersebut karena kemampuannya mengekstraksi fitur secara otomatis dan hierarkis dari data gambar.

Salah satu arsitektur *deep learning* yang telah terbukti memiliki akurasi yang tinggi untuk tugas klasifikasi gambar berbasis *transfer learning* adalah VGG16. VGG16 merupakan arsitektur CNN yang memiliki keunggulan pada kedalaman arsitekturnya yang menggunakan filter konvolusi 3×3 secara konsisten sehingga menghasilkan representasi fitur yang kaya [5]. Untuk meningkatkan performa klasifikasi lebih lanjut, pendekatan hybrid yang menggabungkan CNN sebagai *feature extractor* dengan *Support Vector Machine* (SVM) sebagai *classifier* telah menjadi topik penelitian yang semakin diminati. Pendekatan hybrid VGG16-SVM mampu meningkatkan akurasi klasifikasi gulma multikelas sebesar 2,7% dibandingkan VGG16 dengan *softmax* pada umumnya [5]. Penelitian sebelumnya berhasil mengklasifikasikan spesies bunga pada dataset Flower17 dan Flower102 dengan akurasi hingga 96,39% menggunakan kombinasi fitur dari arsitektur VGG16 yang diklasifikasikan dengan SVM-RBF [4]. Peningkatan ini juga dikaitkan dengan keunggulan SVM yang menggunakan prinsip *Structural Risk Minimization* (SRM) dalam mencegah *overfitting* pada jumlah sampel yang terbatas.

Penerapan dan membandingkan pendekatan klasifikasi citra bunga yang otomatis dan praktis diperlukan untuk mengetahui dan membuktikan teknologi yang *robust* untuk melakukan klasifikasi citra dan tingkat perbedaan hasil akurasi pada masing-masing teknologi. Sehingga, penelitian ini bertujuan untuk menerapkan dan membandingkan dua pendekatan dalam mengklasifikasikan citra bunga. Target objek yang diambil peneliti adalah bunga-bunga dengan jenis yang memiliki kemiripan morfologi, sehingga dapat mengetahui keberhasilan teknologi dengan baik.

1.2 Rumusan Masalah

Adapun rumusan masalah dalam penelitian ini adalah sebagai berikut.

1. Apa saja model machine learning dan deep learning yang memiliki akurasi tinggi dalam melakukan klasifikasi citra?
2. Bagaimana kinerja antara pendekatan *machine learning* dan *deep learning* dalam melakukan klasifikasi citra?

3. Berapa tingkat akurasi pendekatan *machine learning* dan *deep learning* dalam mengklasifikasikan citra?
4. Apa pendekatan teknologi yang menghasilkan performa terbaik antara *machine learning* dan *deep learning* berdasarkan metrik evaluasi *accuracy*, *precision*, *recall*, dan F1-score?

1.3 Batasan Masalah

Adapun batasan masalah dalam penelitian ini adalah sebagai berikut.

1. Dataset yang digunakan terdiri dari empat kelas bunga, yaitu Melati Jakarta, Melati Jepang, Bintaro, dan Tapak Dara, dengan masing-masing kelas berjumlah 360 gambar sehingga total data sebanyak 1.440 gambar.
2. Metode *machine learning* yang digunakan adalah *Support Vector Machine* (SVM) dengan ekstraksi fitur HOG dan *Random Forest*.
3. Metode *deep learning* yang digunakan adalah VGG16, MobileNetV2, hybrid VGG16-SVM dan MobileNetV2-Random Forest.

1.4 Tujuan Penelitian

Adapun tujuan dalam penelitian ini adalah sebagai berikut.

1. Mengetahui model apa saja dari pendekatan *machine learning* dan *deep learning* atau hybrid antar keduanya yang memiliki akurasi tinggi dalam melakukan klasifikasi citra.
2. Mengetahui proses dan kinerja antara pendekatan *machine learning* dan *deep learning* dalam melakukan klasifikasi citra.
3. Membandingkan dan mengetahui tingkat akurasi pendekatan *machine learning* dan *deep learning* dalam mengklasifikasikan citra
4. Mengetahui dan membandingkan pendekatan teknologi yang menghasilkan performa terbaik antara *machine learning* dan *deep learning* berdasarkan metrik evaluasi *accuracy*, *precision*, *recall*, dan F1-score.

1.5 Manfaat Penelitian

Adapun manfaat dalam penelitian ini adalah sebagai berikut.

1. Bagi akademik, penelitian ini memberikan kontribusi berupa perbandingan empiris antara metode *machine learning* dan *deep learning* untuk klasifikasi

gambar, sehingga dapat menjadi referensi bagi penelitian selanjutnya di bidang klasifikasi citra flora.

2. Bagi masyarakat, sistem klasifikasi otomatis yang dikembangkan dapat membantu masyarakat umum, pecinta tanaman, dan tenaga penyuluh pertanian dalam mengidentifikasi jenis bunga secara cepat dan akurat tanpa memerlukan keahlian khusus di bidang botani.
3. Bagi pengembang teknologi, penelitian ini memberikan wawasan tentang efektivitas arsitektur pendekatan *machine learning*, *deep learning*, atau *hybrid* keduanya pada dataset gambar berskala kecil hingga menengah, yang dapat dijadikan dasar pengembangan aplikasi pengenalan tanaman berbasis *mobile* maupun *website*.

BAB II

TINJAUAN PUSTAKA

2.1 Penelitian Terkait

Pada pelaksanaan penelitian ini, peneliti telah melakukan studi literatur pada beberapa penelitian sebelumnya yang memiliki kesamaan dari segi aspek subjek, objek, alat, dan bahan yang digunakan pada penelitian terkait. Hal ini menjadi pendukung penelitian yang akan dilakukan agar lebih terarah.

2.1.1 Model Klasifikasi Bunga

Yanjuan Wu dan kawan-kawan mempublikasikan penelitian berjudul "*Multi-Class Weed Recognition Using Hybrid CNN-SVM Classifier*" dalam jurnal *Sensors* (MDPI), volume 23, nomor 16, artikel 7153. Penelitian ini mengembangkan model pengenalan gulma multikelas berbasis *hybrid* CNN-SVM untuk mendukung pertanian presisi. Penelitian ini menggunakan dua arsitektur CNN *pretrained*, yaitu VGG16 dan ResNet-50, sebagai *feature extractor* dengan bobot awal yang diinisialisasi secara acak (*from scratch*). Setelah proses pelatihan CNN selesai, *fully connected layer* terakhir dihapus sehingga CNN berfungsi sebagai ekstraktor vektor fitur berdimensi tinggi. Vektor fitur yang dihasilkan kemudian digunakan untuk melatih *Support Vector Machine* (SVM) dengan kernel RBF menggunakan strategi *One-Versus-Rest* (OVR) untuk menangani masalah multikelas. Parameter optimal SVM (C dan γ) ditentukan melalui *grid search* dengan 5-fold *cross-validation*. Dataset yang digunakan adalah DeepWeeds, sebuah dataset publik yang terdiri dari 8 kelas gulma dengan total 16.800 gambar. Proses *preprocessing* meliputi *resize* ke 224×224 piksel, augmentasi data (*RandomHorizontalFlip*, *RandomCrop*), normalisasi *ToTensor*, dan standarisasi *Normalize* dengan nilai mean dan standar deviasi ImageNet. Pelatihan menggunakan optimizer Adam dengan *learning rate* 0,00005 dan *batch size* 16 selama 450 *epoch*. Model ResNet-50-SVM mencapai akurasi 97,6%, *precision* 97,7%, *recall* 97,6%, dan *F1-score* 97,6%. Model VGG16-SVM mencapai akurasi 95,9%, *precision* 96,0%, *recall* 95,8%, dan *F1-score* 95,9%. Kedua model hybrid CNN-SVM mengungguli arsitektur CNN *standalone* dengan *softmax*: VGG16 (93,2%), ResNet-50 (96,1%), GoogLeNet (93,6%), DenseNet-121 (94,3%), dan PSO-CNN (96,9%). Peneliti menyimpulkan bahwa pendekatan hybrid CNN-SVM

secara konsisten menghasilkan akurasi yang lebih tinggi dibandingkan CNN dengan *softmax*. Peningkatan ini dikaitkan dengan penggunaan prinsip *Structural Risk Minimization* (SRM) pada SVM yang lebih efektif mencegah *overfitting* dibandingkan *Empirical Risk Minimization* (ERM) yang digunakan *softmax*. Penelitian Wu et al. (2023) menjadi acuan utama dalam penelitian ini, terutama pada pemilihan arsitektur VGG16-SVM, strategi OVR, penggunaan kernel RBF, serta parameter pelatihan. Keterbatasan penelitian ini adalah belum diujikan pada dataset bunga dan tidak membandingkan secara langsung dengan metode *Machine Learning* konvensional [5].

Yeqi Fei dan kawan-kawan mempublikasikan penelitian berjudul "*A Lightweight Attention-Based Convolutional Neural Networks for Fresh-Cut Flower Classification*" dalam jurnal *IEEE Access*, volume 11. Penelitian ini mengembangkan sistem klasifikasi bunga potong segar berbasis CNN ringan (*lightweight*) dengan mekanisme *attention* untuk aplikasi lini produksi industri. Penelitian ini mengusulkan arsitektur berbasis ShuffleNet V2 yang dimodifikasi dengan penambahan mekanisme *attention* untuk meningkatkan fokus model pada fitur-fitur diskriminatif bunga. *Transfer learning* diterapkan dengan bobot *pretrained* dari dataset publik. Data yang digunakan terdiri dari 434 gambar bunga mawar dalam 5 spesifikasi kualitas yang dikumpulkan menggunakan kamera stereo berwarna binocular, menghasilkan gambar 3-kanal RGB dan 4-kanal RGB+*depth*. Untuk mengatasi keterbatasan jumlah sampel, dikembangkan strategi augmentasi data yang komprehensif sesuai kondisi lini produksi nyata, menghasilkan dataset yang lebih bervariasi. Eksperimen membandingkan model yang diusulkan dengan berbagai arsitektur CNN populer seperti MobileNet, ShuffleNet, dan EfficientNet. Model yang diusulkan mencapai akurasi klasifikasi 98,891% pada dataset 3-kanal (RGB) dan 99,915% pada dataset 4-kanal (RGB+*depth*). Kecepatan prediksi mencapai 0,020 detik per bunga, menjadikannya jauh lebih cepat dibandingkan mesin klasifikasi bunga konvensional yang tersedia di pasaran. Model juga unggul dalam jumlah parameter yang lebih sedikit dibandingkan jaringan tanpa mekanisme *attention*. Peneliti menyimpulkan bahwa arsitektur CNN ringan yang dilengkapi mekanisme *attention* mampu mencapai akurasi tinggi sekaligus kecepatan inferensi yang memadai untuk aplikasi industri nyata. Penggabungan informasi *depth* dengan

RGB terbukti meningkatkan akurasi secara signifikan. Penelitian ini memberikan wawasan tentang pentingnya augmentasi data pada dataset kecil serta efektivitas *transfer learning* untuk klasifikasi bunga. Namun, penelitian ini hanya berfokus pada satu jenis bunga (mawar) dalam lingkungan terkontrol dan tidak membandingkan dengan metode *Machine Learning* konvensional maupun pendekatan hybrid CNN-SVM [6].

Samarth Agrawal dan kawan-kawan mempublikasikan penelitian berjudul "*Comparative Analysis of CNN-Based Approaches for Flower Classification*" dalam jurnal *Smart Agricultural Technology* (Elsevier), volume 14, artikel 102213. Penelitian ini melakukan evaluasi komprehensif terhadap berbagai arsitektur CNN modern untuk klasifikasi spesies bunga dan tahap mekar (*bloom stage*) secara simultan. Penelitian ini mengevaluasi lima arsitektur *deep learning*, yaitu VGG16, ResNet50, EfficientNetV2, MobileNetV3, dan backbone YOLOv10 yang direpurposing sebagai *feature extractor* untuk tugas klasifikasi. Seluruh model diuji menggunakan kerangka klasifikasi *dual-output* yang memprediksi spesies bunga (mawar dan bunga matahari) serta empat tahap mekar (*bud*, *partially bloom*, *fully bloom*, dan *wilt*) secara bersamaan dari satu gambar *input*. Eksperimen dilakukan pada dataset bunga mawar dan bunga matahari dengan empat tahap mekar yang berbeda. *Transfer learning* diterapkan pada semua arsitektur yang diuji. Untuk tugas klasifikasi spesies bunga, semua model menunjukkan performa yang relatif baik. Namun untuk tugas klasifikasi tahap mekar yang lebih menantang, VGG16 mencapai akurasi tertinggi sebesar 82,8%, sementara ResNet50 menunjukkan performa yang stabil dan seimbang pada kedua tugas. MobileNetV3 memberikan akurasi moderat sebagai alternatif ringan, sedangkan EfficientNetV2 dan YOLOv10 menunjukkan performa yang lebih rendah pada prediksi tahap mekar. Peneliti menyimpulkan bahwa VGG16 merupakan arsitektur yang paling andal untuk klasifikasi visual bunga, khususnya ketika perbedaan antarkelas bersifat halus seperti pada tahap mekar bunga. ResNet50 menawarkan keseimbangan yang baik antara akurasi dan kestabilan pada kedua tugas klasifikasi. Penelitian ini memperkuat pilihan VGG16 sebagai *backbone* dalam penelitian ini karena kemampuannya menangkap perbedaan visual halus antarkelas bunga. Keterbatasan penelitian adalah hanya menggunakan dua spesies bunga dan tidak membandingkan

dengan metode *Machine Learning* konvensional maupun pendekatan hybrid CNN-SVM pada dataset bunga tropis [7].

2. 2 Landasan Teori

2. 2. 1 Klasifikasi Citra Digital

Klasifikasi citra digital merupakan salah satu tugas fundamental dalam bidang *computer vision* yang bertujuan untuk memetakan sebuah gambar ke dalam satu atau lebih kategori kelas yang telah ditentukan sebelumnya. Dalam konteks klasifikasi bunga, sistem diberikan masukan berupa gambar RGB dan diharapkan mampu mengidentifikasi jenis bunga yang terdapat pada gambar tersebut secara otomatis. Tantangan utama dalam klasifikasi citra bunga meliputi tingginya variasi intra-kelas akibat perbedaan sudut pandang, kondisi pencahayaan, dan tahap mekar, serta kesamaan inter-kelas antara spesies yang memiliki morfologi serupa [7]. Pendekatan klasifikasi citra secara umum terbagi menjadi dua paradigma, yaitu pendekatan berbasis fitur manual (*handcrafted features*) yang digunakan pada metode *Machine Learning* konvensional, dan pendekatan berbasis pembelajaran fitur otomatis (*learned features*) yang digunakan pada metode *Deep Learning*.

2. 2. 2 Support Vector Machine (SVM)

Support Vector Machine (SVM) adalah algoritma klasifikasi yang bekerja dengan mencari *hyperplane* optimal yang memisahkan data dari kelas yang berbeda dengan *margin* (jarak) maksimum antara *hyperplane* dan titik-titik data terdekat dari masing-masing kelas yang disebut *support vectors*. Optimasi SVM diformulasikan sebagai permasalahan pemrograman kuadratik (*quadratic programming*) yang bertujuan meminimalkan fungsi objektif berikut [5].

$$\min_{\omega, b, \xi} \frac{1}{2} (\omega^i)^T \omega^i + C \sum_{j=1}^l \xi_j^i$$

Di mana ω adalah vektor normal yang menentukan arah *hyperplane*, b adalah bias term, C adalah parameter penalti yang mengontrol *trade-off* antara margin dan kesalahan klasifikasi, dan ξ_j^i adalah *hinge loss function* yang mengukur pelanggaran margin.

Keunggulan fundamental SVM dibandingkan *softmax* pada CNN adalah penggunaan prinsip *Structural Risk Minimization* (SRM) yang secara bersamaan meminimalkan *empirical risk* dan membatasi kompleksitas model (*VC dimension*), sehingga lebih efektif mencegah *overfitting* terutama pada dataset yang kecil. Sebagai perbandingan, *softmax* hanya menggunakan *Empirical Risk Minimization* (ERM) yang murni meminimalkan kesalahan pada data pelatihan tanpa pengendalian kompleksitas model [5].

Untuk menangani data yang tidak dapat dipisahkan secara linear, SVM menggunakan fungsi *kernel* yang memetakan data ke ruang berdimensi lebih tinggi secara implisit. Dalam penelitian ini digunakan kernel *Radial Basis Function* (RBF) yang didefinisikan sebagai berikut.

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Di mana γ adalah parameter kernel yang menentukan lebar distribusi Gaussian. Parameter C dan γ ditentukan melalui grid search dengan rentang $C \in \{2^{-3}, 2^{-2}, \dots, 2^3\}$ dan $\gamma \in \{2^{-5}, 2^{-4}, \dots, 2^3\}$ menggunakan 5-fold cross-validation [7]. Untuk menangani masalah klasifikasi multikelas (4 kelas bunga), digunakan strategi One-Versus-Rest (OVR) di mana sebanyak n classifier SVM dibangun secara independen. Setiap classifier ke- i memisahkan kelas ke- i dari semua kelas lainnya. Pada saat pengujian, kelas dengan nilai fungsi keputusan tertinggi dipilih sebagai prediksi akhir.

$$\hat{y} = \arg \max_i [(\omega^i)^T \phi(x) + b^i].$$

2.2.3 Random Forest

Random Forest adalah algoritma *ensemble learning* yang bekerja dengan membangun sejumlah pohon keputusan (*decision tree*) secara paralel pada *subset* data pelatihan dan *subset* fitur yang dipilih secara acak, kemudian menggabungkan prediksi seluruh pohon melalui mekanisme voting mayoritas untuk klasifikasi [2]. Setiap pohon dalam *Random Forest* dibangun menggunakan teknik *bootstrap aggregating* (bagging), di mana setiap pohon dilatih pada sampel data yang diambil dengan pengembalian dari dataset asli. Pada setiap node percabangan, hanya sebagian fitur yang dipilih secara acak (*max_features*) untuk dipertimbangkan

sebagai kandidat pemisah terbaik, sehingga menghasilkan pohon-pohon yang beragam dan saling melengkapi.

Keunggulan *Random Forest* dibandingkan pohon keputusan tunggal meliputi ketahanan yang lebih baik terhadap *overfitting*, kemampuan menangani data berdimensi tinggi, tidak memerlukan normalisasi data, serta menghasilkan estimasi kepentingan fitur (*feature importance*) secara alami. Dalam penelitian ini, *Random Forest* digunakan sebagai metode *machine learning baseline* dengan parameter utama: jumlah pohon ($n_estimators = 200$), strategi fitur $max_features = 'sqrt'$, dan pembobotan kelas $class_weight = 'balanced'$ untuk mengantisipasi ketidakseimbangan kelas [2].

2. 2. 4 Histogram of Oriented Gradients (HOG)

Histogram of Oriented Gradients (HOG) adalah deskriptor fitur merepresentasikan informasi bentuk dan tekstur objek dalam gambar melalui distribusi arah gradien intensitas piksel. HOG bekerja dengan membagi gambar menjadi sel-sel kecil (cells) berukuran $p \times p$ piksel, kemudian menghitung histogram orientasi gradien pada setiap sel dengan jumlah bin orientasi tertentu. Sel-sel tersebut kemudian dikelompokkan menjadi blok-blok yang saling tumpang tindih (*overlapping blocks*) dan dinormalisasi untuk meningkatkan invariansi terhadap perubahan pencahayaan dan kontras. Dalam penelitian ini, parameter HOG yang digunakan adalah 9 orientasi ($orientations = 9$), ukuran sel 8×8 piksel ($pixels_per_cell = (8, 8)$), dan ukuran blok 2×2 sel ($cells_per_block = (2, 2)$) pada gambar berukuran 64×64 piksel yang terlebih dahulu dikonversi ke *grayscale*. Konfigurasi ini menghasilkan vektor fitur HOG berdimensi 1.764. Vektor fitur HOG kemudian digabungkan dengan 6 fitur statistik warna berupa *mean* dan standar deviasi masing-masing kanal R, G, dan B, sehingga total dimensi vektor fitur yang digunakan sebagai masukan *Random Forest* adalah 1.770 dimensi.

2. 2. 5 Convolutional Neural network (CNN)

Convolutional Neural Network (CNN) adalah kelas arsitektur *deep learning* yang dirancang khusus untuk memproses data berbentuk *grid*, seperti gambar digital. CNN memperkenalkan tiga mekanisme utama yang membedakannya dari jaringan saraf biasa, yaitu lapisan konvolusi (*convolutional layer*), lapisan *pooling*,

dan *weight sharing*. Lapisan konvolusi mengaplikasikan sejumlah filter (kernel) yang dipelajari selama pelatihan pada peta fitur masukan untuk menghasilkan peta aktivasi yang merepresentasikan kehadiran pola-pola tertentu, seperti tepi, tekstur, dan bentuk [8]. Secara hierarkis, lapisan-lapisan awal CNN mendeteksi fitur primitif berupa tepi dan gradien, lapisan tengah menangkap pola dan tekstur yang lebih kompleks, sedangkan lapisan dalam merepresentasikan konsep semantik tingkat tinggi seperti bagian objek dan keseluruhan objek. Keunggulan CNN dalam klasifikasi citra terletak pada kemampuannya mengekstraksi fitur yang relevan secara otomatis langsung dari piksel gambar tanpa memerlukan rekayasa fitur manual, serta kemampuan berbagi bobot (*weight sharing*) yang secara signifikan mengurangi jumlah parameter yang harus dipelajari dibandingkan jaringan *fully connected* [7].

2. 2. 6 VGG16

VGG16 adalah arsitektur CNN yang memiliki karakteristik utama pada penggunaan filter konvolusi berukuran 3×3 secara konsisten di seluruh jaringan dengan $\text{stride} = 1$ dan $\text{padding} = 1$, disertai lapisan max pooling 2×2 dengan $\text{stride} = 2$ sebagai pemisah antar-blok konvolusi. Penggunaan filter 3×3 yang bertumpuk memiliki *receptive field* yang ekuivalen dengan filter berukuran lebih besar namun dengan parameter yang jauh lebih sedikit dan *nonlinearitas* yang lebih banyak. Arsitektur VGG16 terdiri dari lima blok konvolusi dengan kedalaman kanal berturut-turut 64, 128, 256, 512, dan 512, diikuti tiga lapisan *fully connected* berukuran 4.096, 4.096, dan 1.000 (*softmax*) [9]. Dalam penelitian ini, VGG16 digunakan sebagai *feature extractor* dengan mengganti lapisan *classifier* terakhir (fc 1.000) dengan lapisan baru sesuai jumlah kelas (4 kelas). Setelah pelatihan VGG16, lapisan *classifier* terakhir dihapus sehingga keluaran jaringan berupa vektor fitur berdimensi 4.096 dari lapisan *fully connected* keempat (fc4096), yang kemudian digunakan sebagai masukan untuk pelatihan SVM [5].

2. 3 MobileNetV2

MobileNetV2 merupakan salah satu arsitektur *Convolutional Neural Network* (CNN) *state-of-the-art* yang dirancang secara khusus sebagai model jaringan yang ringan (*lightweight network*) [6]. Keunggulan utama dari arsitektur

MobileNetV2 terletak pada efisiensinya, di manamodel ini memiliki jumlah parameter yang sedikit serta kecepatan performa komputasi yang tinggi, sehingga sangat ideal untuk diimplementasikan pada aplikasi perangkat seluler (*mobile applications*) maupun perangkat dengan sumber daya terbatas yang membutuhkan pemrosesan *real-time* [10].

Secara teknis, model standar MobileNetV2 dirancang untuk memproses data masukan berupa citra berwarna (RGB) dengan ukuran resolusi seragam, yaitu 224×224 piksel. Dalam berbagai penelitian dan aplikasi praktis, model ini sering digunakan dengan pendekatan *transfer learning*, di mana model yang telah dilatih sebelumnya (seperti pada dataset besar ImageNet) disesuaikan (*fine-tuning*) untuk mempercepat tugas klasifikasi pada dataset baru [9]. Lebih jauh lagi, selain dapat menghasilkan prediksi secara langsung, MobileNetV2 juga terbukti sangat andal jika difungsikan murni sebagai alat pengekstraksi fitur mendalam (*deep feature extractor*). Sebagai contoh, model ini mampu mengekstraksi 1.000 vektor fitur dari lapisan *Logits*-nya, yang kemudian dapat diproses lebih lanjut atau diklasifikasikan menggunakan algoritma *machine learning* tradisional seperti *Support Vector Machine* (SVM) untuk mengoptimalkan akurasi klasifikasi akhir [10]

2. 4 *Transfer Learning*

Transfer learning adalah teknik dalam *deep learning* yang memanfaatkan pengetahuan yang telah diperoleh model dari pelatihan pada satu tugas atau domain tertentu untuk diterapkan pada tugas atau domain yang berbeda namun berkaitan. Dalam konteks klasifikasi citra, model CNN yang telah dilatih pada dataset berskala besar seperti ImageNet (1,2 juta gambar, 1.000 kelas) dapat dimanfaatkan sebagai titik awal untuk melatih model pada dataset yang lebih kecil dan spesifik [9].

Terdapat dua pendekatan utama dalam transfer learning, yaitu pertama *feature extraction*, di mana seluruh bobot lapisan konvolusi dibekukan (*frozen*) dan hanya lapisan klasifikasi baru yang dilatih dan kedua *fine-tuning*, di mana sebagian atau seluruh bobot model *update* selama pelatihan pada dataset target. Pendekatan transfer learning sangat relevan dalam penelitian ini karena dataset yang tersedia (1.440 gambar) tergolong kecil untuk melatih CNN dari awal (*from scratch*), sehingga pemanfaatan bobot *pretrained* ImageNet pada VGG16 memungkinkan

model memanfaatkan representasi fitur visual umum yang telah dipelajari dari jutaan gambar [9].

BAB III

METODOLOGI PENELITIAN

3.1 Dataset

Dataset yang digunakan dalam penelitian ini adalah dataset gambar bunga yang dikumpulkan secara mandiri (*self-collected dataset*) oleh peneliti. Dataset terdiri dari empat kelas bunga yang umum dijumpai di lingkungan perkotaan Indonesia. Dataset ini diberi nama "Dataset Bunga Lokal Putih", merupakan kumpulan gambar empat jenis bunga berwarna putih, yaitu Melati Jakarta (*Jasminum sambac*), Melati Jepang (*Pseuderanthemum reticulatum*), Bintaro (*Cerbera manghas*), dan Tapak Dara Putih (*Catharanthus roseus*). Dataset dikumpulkan secara mandiri oleh peneliti di lapangan melalui empat cara, yaitu pengambilan foto pada latar belakang abstrak dan jarak 10 cm, pengambilan foto pada latar belakang abstrak dan jarak 5 cm, pengambilan foto pada latar belakang polos dan jarak 10 cm, dan pengambilan foto pada latar belakang polos dan jarak 5 cm. Keempat cara pengumpulan ini dilakukan untuk memastikan variasi data yang cukup, meliputi perbedaan sudut pandang (*angle*), latar belakang (*background*), serta jarak kepada bunga. Variasi ini penting agar model yang dilatih memiliki kemampuan generalisasi yang baik terhadap kondisi nyata di lapangan [1].

Dataset terdiri dari 1.440 citra gambar yang terbagi menjadi 4 kelas sebagai berikut.

No.	Kelas Bunga	Nama Ilmiah	Jumlah gambar
1	Melati Jakarta	<i>Jasminum sambac</i>	360
2	Melati Jepang	<i>Pseuderanthemum reticulatum</i>	360
3	Bintaro	<i>Cerbera manghas</i>	360
4	Tapak Dara	<i>Catharanthus roseus</i>	360
Total			1.440

Pada satu kelas, dataset terdiri dari 40 citra asli dan 320 citra hasil augmentasi dengan beberapa teknik augmentasi meliputi horizontal flip, rotasi 90°, penurunan kecerahan, dan peningkatan kecerahan. Distribusi data yang seimbang (*balanced dataset*) antarkelas memastikan tidak adanya bias kelas (*class*

imbalance) selama proses pelatihan model, sehingga model tidak cenderung memprediksi kelas tertentu secara berlebihan [5]. Label merupakan kelas jenis bunga yang menjadi target prediksi model. Terdapat empat kategori label pada penelitian ini, yaitu Melati Jakarta, Melati Jepang, Bintaro, dan Tapak Dara. Setiap gambar memiliki tepat satu label sehingga penelitian ini merupakan permasalahan klasifikasi multikelas dengan satu label (*multi-class single-label classification*).



Data yang digunakan dalam penelitian ini adalah data gambar (*image data*) berformat JPEG dan PNG berwarna. Sebelum digunakan, seluruh gambar distandarisasi ke ukuran seragam menjadi 224×224 piksel. Jumlah fitur yang digunakan bergantung pada metode yang diterapkan, karena masing-masing metode menggunakan pendekatan ekstraksi fitur yang berbeda. Pada metode *Random Forest* (ML), fitur diekstraksi secara manual menggunakan deskriptor *Histogram of Oriented Gradients* (HOG) dengan konfigurasi 9 orientasi, ukuran sel 8×8 piksel, dan ukuran blok 2×2 sel pada gambar *grayscale* berukuran 64×64 piksel, menghasilkan vektor fitur HOG berdimensi 1.764. Vektor ini kemudian digabungkan dengan 6 fitur statistik warna berupa nilai *mean* dan standar deviasi dari masing-masing kanal R, G, dan B, sehingga total dimensi fitur masukan *Random Forest* adalah 1.770 dimensi. Pada metode MobileNetV2 (DL), fitur diekstraksi secara otomatis oleh lapisan konvolusi MobileNetV2 melalui

mekanisme *depthwise separable convolution*. Setelah melewati lapisan *Global Average Pooling* (GAP), representasi fitur yang dihasilkan berdimensi 1.280 dimensi yang kemudian diteruskan ke *classifier head* untuk klasifikasi. Pada metode VGG16 (DL), fitur diekstraksi dari lapisan *fully connected* keempat (fc4096) VGG16 menghasilkan vektor berdimensi 4.096 dimensi yang digunakan sebagai masukan SVM. Pada metode VGG16+SVM dan MobileNetV2+RF, fitur yang diekstraksi dari lapisan CNN digunakan sebagai masukan untuk *classifier* eksternal (SVM atau RF), masing-masing dengan dimensi 4.096 dan 1.280 sesuai arsitektur CNN yang digunakan [7].

3.2 *Preprocessing Data*

Tahapan *preprocessing* data dalam penelitian ini dibedakan berdasarkan metode yang digunakan, karena setiap metode memiliki kebutuhan representasi *input* yang berbeda. Secara umum, *pipeline preprocessing* mencakup tahapan berikut.

a. *Load dan Resize Gambar*

Seluruh dataset dimuat dari folder yang terorganisasi per kelas (Melati_Jakarta, Melati_Jepang, Bintaro, Tapak_Dara), kemudian dilakukan konversi mode warna ke RGB menggunakan PIL Image. Selanjutnya, dataset dilakukan *resize* untuk menyeragamkan dimensi input. Ukuran *resize* disesuaikan dengan kebutuhan masing-masing metode, 224×224 piksel untuk model *deep learning* MobileNetV2 dan VGG16 mengikuti ukuran input standar arsitektur tersebut, sedangkan 64×64 piksel digunakan untuk metode *machine learning* klasik *Random Forest* untuk mereduksi dimensi dan mempercepat komputasi. Khusus untuk SVM dengan ekstraksi HOG secara mandiri, *resize* dilakukan ke ukuran 224×224 piksel menggunakan interpolasi cv2.INTER_AREA yang optimal untuk operasi *downscale*.

b. *Preprocessing Khusus untuk SVM + HOG*

Terdapat dua tahapan tambahan yang diterapkan untuk meningkatkan kualitas citra, sebelum ekstraksi fitur HOG pada model SVM. Pertama, gambar dikonversi dari BGR ke *Grayscale* menggunakan cv2.cvtColor, karena HOG bekerja pada distribusi gradien intensitas sehingga informasi warna tidak diperlukan. Kedua, diterapkan CLAHE (*Contrast Limited Adaptive Histogram*

Equalization) dengan `clipLimit=2.0` dan `tileGridSize=(16,16)` untuk meningkatkan kontras lokal secara adaptif. Setiap tile berukuran $224/16 = 14 \times 14$ piksel, memberikan pemerataan kontras yang representatif di seluruh gambar. Ketiga, dilakukan *Gaussian Blur* dengan kernel (5×5) untuk meredam *noise salt-and-pepper*, dengan ukuran kernel yang diperbesar dari standar (3×3) karena resolusi gambar 224×224 memerlukan efek pelemahan yang lebih proporsional.

c. Ekstraksi Fitur HOG

Pada metode SVM, dilakukan ekstraksi fitur Histogram of Oriented Gradients (HOG) untuk mengubah citra menjadi vektor fitur numerik yang dapat diproses oleh algoritma klasifikasi. Parameter HOG yang digunakan adalah 9 bin orientasi (0° – 180°), `cells_per_block (2,2)`, dan normalisasi blok L2-Hys yang robust terhadap perubahan pencahayaan. Pada model SVM mandiri, diuji dua konfigurasi `pixels_per_cell (14,14)` yang menghasilkan vektor fitur sekitar 11.664 dimensi lebih detail dan `(16,16)` yang menghasilkan sekitar 6.084 dimensi lebih efisien secara komputasi..

d. Normalisasi Fitur (Machine Learning)

Khusus untuk model SVM, normalisasi fitur dilakukan menggunakan *StandardScaler* (`mean = 0`, `std = 1`) yang diintegrasikan ke dalam *Pipeline sklearn* bersama model SVC. Normalisasi ini wajib dilakukan sebelum SVM karena vektor HOG pada gambar 224×224 memiliki variasi nilai yang besar. Pada model *Random Forest*, normalisasi serupa diterapkan dengan *StandardScaler* yang memilih 25% fitur paling informatif dari total vektor fitur gabungan.

e. *Preprocessing* Normalisasi untuk *Deep Learning*

Pada model MobileNetV2 dan VGG16, normalisasi piksel dilakukan menggunakan fungsi *preprocess_input* bawaan masing-masing arsitektur dari Keras. MobileNetV2 menggunakan *preprocess_input* yang menormalisasi nilai piksel ke rentang $[-1, 1]$, sementara VGG16 menggunakan *preprocess_input* yang melakukan *zero-centering* berdasarkan *mean* kanal *ImageNet*. Label dikodekan menggunakan *one-hot encoding via to_categorical* menjadi vektor biner berdimensi 4 (jumlah kelas).

f. *Data Augmentation*

Augmentasi data diterapkan hanya pada data *training*, tidak pada data validasi maupun testing, untuk menghindari data leakage dan mencegah *overfitting*. Augmentasi dilakukan secara *on-the-fly* menggunakan *ImageDataGenerator* dari Keras. Untuk model MobileNetV2, parameter augmentasi yang digunakan adalah horizontal flip, rotasi $\pm 45^\circ$, zoom 80–120%, translasi horizontal dan vertikal sebesar 10%, serta variasi kecerahan [0.8, 1.2]. Untuk model VGG16, augmentasi lebih agresif meliputi horizontal flip, rotasi hingga 90° , zoom 20%, translasi 15%, variasi kecerahan [0.7, 1.3], dan shear transform 10%, mengacu pada Fei et al. (2023). Data validasi dan testing menggunakan *ImageDataGenerator* tanpa augmentasi apapun.

g. *Pembagian Data*

Pembagian dataset dilakukan secara stratified (*stratified split*) untuk memastikan proporsi kelas seimbang di setiap subset. Rasio pembagian yang digunakan bervariasi antarmodel sesuai dengan jurnal rujukan. Model SVM menggunakan 70% training dan 30% testing (Percobaan 1) serta 80% training dan 20% testing (Percobaan 2 dan 3), Random Forest dan MobileNetV2 menggunakan 70% training, 10% validasi, dan 20% testing, sedangkan VGG16 menggunakan 70% training, 15% validasi, dan 15% testing. Pembagian dua tahap dilakukan secara berurutan, pertama memisahkan data testing, kemudian memisahkan validasi dari sisa data, dengan `random_state = 42` untuk reproduktibilitas.

3.3 Metode Machine Learning dari UTS

Penelitian ini menggunakan dua metode *machine learning* sebagai *baseline* dari UTS, yaitu *Support Vector Machine* (SVM) dengan ekstraksi fitur HOG, dan Random Forest dengan kombinasi fitur HOG dan fitur warna. Pada *notebook* `Model_VGG16_SVM.ipynb`, *Random Forest* juga berfungsi sebagai pembanding langsung terhadap metode *hybrid* CNN-SVM yang dikembangkan di UAS.

3.3.1 *Support Vector Machine* (SVM)

Metode *machine learning* yang digunakan pada UTS adalah *Support Vector Machine* (SVM) dengan ekstraksi fitur *Histogram of Oriented Gradients* (HOG).

SVM dipilih karena terbukti efektif untuk masalah klasifikasi citra bunga dengan data berlabel terbatas. SVM dengan fitur yang diekstraksi dari CNN pra-latih menghasilkan akurasi hingga 96,39% pada Flower17 dan 95,70% pada Flower102, melampaui metode deep learning end-to-end pada skala dataset yang kecil [4]. SVM bekerja dengan mencari *hyperplane* pemisah optimal bermargin maksimum antarspesies bunga, sehingga sangat cocok untuk ruang fitur berdimensi tinggi seperti vektor HOG. Selain itu, SVM tidak memerlukan data berlabel dalam jumlah sangat besar untuk mencapai generalisasi yang baik, menjadikannya pilihan tepat untuk dataset 1.440 gambar penelitian ini. Pipeline SVM yang diimplementasikan dalam `svm_klasifikasi_bunga.py` mengikuti alur (1) load gambar dan resize ke 224×224 piksel menggunakan interpolasi `INTER_AREA`, (2) konversi ke grayscale, (3) peningkatan kontras dengan CLAHE (`clipLimit=2.0`, `tileGridSize=16 \times 16`), (4) pengurangan noise dengan Gaussian Blur kernel (5×5) , (5) ekstraksi fitur HOG, (6) normalisasi fitur dengan `StandardScaler`, (7) pencarian hyperparameter terbaik via `GridSearchCV` 5-fold, dan (8) pelatihan `SVC` serta evaluasi pada data testing.

Penelitian ini menguji tiga konfigurasi percobaan dengan variasi `pixels_per_cell` dan rasio split data. Percobaan pertama menggunakan pixel per cell (14,14), cells per block (2,2), orientations 9, dimensi fitur 11.664, dan split data 70:30. Percobaan kedua menggunakan pixel per cell (14,14), cells per block (2,2), orientations 9, dimensi fitur 11.664, dan split data 80:20. Percobaan ketiga menggunakan pixel per cell (16,16), cells per block (2,2), orientations 9, dimensi fitur 6.084, dan split data 80:20. Pembagian data dilakukan secara stratified (`stratify=labels`) dengan `random_state = 42` untuk reproduktibilitas. Percobaan 1 menggunakan split 70% training dan 30% testing, percobaan 2 dan 3 menggunakan 80% training dan 20% testing. Cross-validation 5-fold kemudian dijalankan pada model terbaik menggunakan seluruh fitur yang tersedia untuk mengukur generalisasi secara lebih robust.

Adapun pelatihan model dilakukan menggunakan Pipeline `sklearn` yang mengintegrasikan `StandardScaler` dan `SVC`. Pencarian hyperparameter terbaik dilakukan dengan `GridSearchCV` 5-fold cross-validation dengan ruang pencarian berikut.

Parameter	Nilai yang Diuji
kernel	['rbf', 'linear']
C	[0.1, 1, 10, 100]
gamma	['scale', 0.01, 0.001]

Total kombinasi yang dievaluasi $2 \times 4 \times 3 = 24$ kombinasi $\times 5$ fold = 120 model. Parameter `decision_function_shape='ovo'` (one-vs-one) digunakan untuk menangani masalah multi-kelas 4 kelas, dan `probability=True` diaktifkan untuk menghasilkan probabilitas prediksi yang dibutuhkan pada demo prediksi.

3.3.2 Random Forest

Random Forest sebuah algoritma *ensemble* berbasis kumpulan *decision tree* yang bekerja dengan mekanisme *bagging* (bootstrap aggregating). *Random Forest* dipilih karena beberapa alasan. Pertama, algoritma ini tahan terhadap *overfitting* karena prediksi final dihasilkan dari voting mayoritas ratusan pohon yang masing-masing dilatih pada subset data dan fitur yang berbeda. Kedua, *Random Forest* tidak sensitif terhadap *outlier* dan mampu menangani fitur berdimensi tinggi dengan baik; ketiga, algoritma ini menyediakan informasi *feature importance* yang berguna untuk menganalisis fitur HOG dan warna mana yang paling berkontribusi dalam membedakan spesies bunga. Peneliti mencatat bahwa kombinasi CNN sebagai ekstraktor fitur dengan *Random Forest* sebagai pengklasifikasi dapat menghasilkan akurasi hingga 99,8% pada klasifikasi bunga, memperkuat relevansi penggunaan RF dalam penelitian ini [6].

Pipeline Random Forest mengikuti alur (1) load gambar dan *resize* ke 64×64 piksel menggunakan PIL, (2) konversi ke *grayscale* via `skimage.color.rgb2gray`, (3) ekstraksi fitur HOG dengan `pixels_per_cell=(8,8)`, `cells_per_block=(2,2)`, `orientations=9`, menghasilkan vektor fitur panjang dari gambar 64×64 , (4) ekstraksi fitur warna berupa *mean* dan standar deviasi tiap kanal R, G, B yang dinormalisasi ke $[0,1]$, menghasilkan 6 nilai tambahan per gambar, (5) penggabungan vektor HOG dan fitur warna menjadi satu vektor fitur kombinasi, (6) normalisasi dengan *StandardScaler*, dan (7) pelatihan *RandomForestClassifier*. Adapun parameter model yang digunakan pada model sebagai berikut.

Parameter	Nilai	Keterangan
-----------	-------	------------

n_estimators	200	Jumlah pohon keputusan dalam hutan
max_depth	None	Pohon tumbuh hingga semua daun murni
min_samples_split	2	Minimum sampel untuk membelah node
min_samples_leaf	1	Minimum sampel di setiap daun
max_features	sqrt	Fitur yang dipertimbangkan tiap split ($\sqrt{\text{total fitur}}$)
class_weight	balanced	Menangani ketidakseimbangan kelas
random_state	42	Reproduktibilitas
n_jobs	-1	Menggunakan semua core CPU

3.4 Pengembangan *Deep Learning* pada UAS

Pada UAS dikembangkan dua model *deep learning* berbasis *transfer learning*, yaitu MobileNetV2 dan VGG16. Keduanya merupakan arsitektur CNN (*Convolutional Neural Network*) pra-latih (*pretrained*) dengan bobot ImageNet yang diterapkan menggunakan pendekatan *transfer learning* berbasis *model-based fine-tuning*. MobileNetV2 berfungsi sebagai model ringan (*lightweight*) yang berfokus pada efisiensi komputasi, sementara VGG16 berfungsi sebagai model berkapasitas tinggi (*high-capacity*) yang mengutamakan kedalaman arsitektur. Perbandingan keduanya memungkinkan analisis *trade-off* antara efisiensi dan performa dalam konteks klasifikasi bunga. Kemudian, dibandingkan performanya satu sama lain serta dengan metode *machine learning* dari UTS.

MobileNetV2 dipilih berdasarkan pada penelitian sebelumnya yang menggunakan *fine-tuned* MobileNetV2 untuk klasifikasi daun grapevine dan mencapai akurasi 97,60% [10]. Secara arsitektural, MobileNetV2 menggunakan *depthwise separable convolution* dan blok residual *inverted* yang membuatnya jauh lebih ringan dibandingkan model konvensional namun tetap kompetitif dalam akurasi. MobileNetV2 merupakan salah satu arsitektur pilihan untuk aplikasi real-time karena keunggulannya dalam efisiensi penyimpanan dan waktu *training* [11]. Hal ini sangat relevan mengingat dataset penelitian ini relatif kecil (1.440 gambar, 4 kelas), sehingga model yang lebih ringan dengan kapasitas generalisasi yang baik menjadi pilihan tepat.

VGG16 dipilih berdasarkan penelitian sebelumnya yang menyebutkan bahwa VGG merupakan arsitektur klasik yang paling umum digunakan untuk transfer learning dalam klasifikasi gambar [6]. Dalam studi perbandingan model CNN untuk klasifikasi bunga menemukan bahwa VGG16 mencapai akurasi 100% untuk klasifikasi spesies bunga dan 82,86% untuk klasifikasi tahap mekar, menjadikannya model terbaik di antara lima arsitektur yang diuji [7]. VGG16 dipilih karena kemampuannya mengekstraksi fitur hierarkis yang kaya, mencakup fitur global bentuk bunga maupun fitur lokal detail tekstur kelopak secara bersamaan.

MobileNetV2 terdiri dari backbone dengan 153 layer, yang di dalamnya mencakup 17 blok *inverted residual* dengan *depthwise separable convolution*, ditambah 7 layer kustom pada *classification head* (GlobalAveragePooling2D, BatchNormalization, Dense-256, Dropout, Dense-128, Dropout, dan Dense-4). Pada Phase 1, seluruh layer backbone dibekukan sehingga hanya 7 layer head yang dilatih. Pada Phase 2 (*fine-tuning*), 50 layer terakhir backbone dibuka (*unfreeze*) dan dilatih ulang bersama head, sehingga total layer yang dilatih meningkat secara signifikan.

VGG16 terdiri dari backbone dengan 19 layer (13 lapisan konvolusional dalam 5 blok + 3 lapisan *fully connected* yang digantikan), ditambah 7 layer *classification head* kustom. Pada Phase 1, seluruh backbone dibekukan. Pada Phase 2, hanya 4 layer terakhir backbone (blok konvolusional Conv5: block5_conv1, block5_conv2, block5_conv3, dan block5_pool) yang dibuka dan dilatih ulang untuk menyesuaikan representasi fitur level tinggi dengan karakteristik visual dataset bunga lokal.

Kedua model menggunakan kombinasi fungsi aktivasi sebagai berikut. ReLU (Rectified Linear Unit, $f(x) = \max(0, x)$) digunakan pada seluruh lapisan *fully connected* tersembunyi (Dense-256, Dense-128 pada MobileNetV2, Dense-1024, Dense-256 pada VGG16). ReLU dipilih karena mengatasi masalah *vanishing gradient*, mempercepat konvergensi, dan merupakan standar *de facto* dalam arsitektur CNN modern. Softmax digunakan pada lapisan output (Dense-4) untuk menghasilkan distribusi probabilitas atas 4 kelas bunga, di mana jumlah probabilitas seluruh kelas selalu sama dengan 1. Fungsi ini sesuai untuk masalah

klasifikasi multi-kelas yang saling eksklusif. Selain itu, fungsi aktivasi internal pada blok MobileNetV2 menggunakan ReLU6 ($f(x) = \min(\max(0, x), 6)$) yang merupakan karakteristik bawaan arsitektur MobileNetV2 untuk efisiensi pada perangkat dengan presisi terbatas. Kedua model menggunakan Adam (*Adaptive Moment Estimation*) sebagai *optimizer*, namun dengan nilai *learning rate* berbeda pada setiap fase. Adam dipilih karena menggabungkan keunggulan AdaGrad (adaptif terhadap frekuensi parameter) dan RMSProp (adaptif terhadap skala gradien), sehingga efisien dan stabil untuk dataset berukuran kecil hingga menengah. Fungsi *loss* yang digunakan adalah *categorical cross-entropy*, sesuai untuk klasifikasi multi-kelas dengan label *one-hot*.

Pelatihan kedua model dilakukan dalam dua fase dengan konfigurasi epoch sebagai berikut.

a. Mobile Net

- Phase 1 (feature extraction, backbone frozen) maksimum 20 epoch.
- Phase 2 (fine-tuning 50 layer terakhir) maksimum 10 epoch.
- Total epoch maksimum 30 epoch.

b. VGG16

- Phase 1 (head training, backbone frozen): maksimum 15 epoch
- Phase 2 (fine-tuning blok Conv5, 4 layer terakhir): maksimum 10 epoch
- Total epoch maksimum: 25 epoch

Pada kedua model, pelatihan aktual dapat berhenti lebih awal dari batas maksimum karena penerapan *callback* EarlyStopping dengan *patience* = 5 yang memantau *val_accuracy*. Jika akurasi validasi tidak meningkat selama 5 epoch berturut-turut, pelatihan dihentikan dan bobot terbaik dipulihkan (*restore_best_weights* = True).

MobileNetV2 menggunakan *batch_size* = 32 dengan 1.008 sampel training (70% dari 1.440), setiap epoch terdiri dari 31 *steps* ($1.008 \div 32 = 31,5$). VGG16 menggunakan *batch_size* = 16, disesuaikan dengan ukuran dataset ~1.440 gambar agar memori GPU tidak kewalahan mengingat VGG16 memiliki ukuran model yang jauh lebih besar dibanding MobileNetV2. Dengan jumlah sampel training sekitar 712 (70% dari 1.020 sampel non-test), setiap epoch terdiri dari sekitar 44 *steps*.

Kedua model menggunakan strategi *learning rate* bertahap untuk mendukung stabilitas *fine-tuning*.

a. MobileNetV2

- Phase 1: $lr = 0.001$ (1×10^{-3}), cukup besar untuk melatih head dari nol.
- Phase 2: $lr = 0.0001$ (1×10^{-4}), diperkecil $10 \times$ agar bobot backbone tidak terganggu terlalu drastis saat *fine-tuning*.

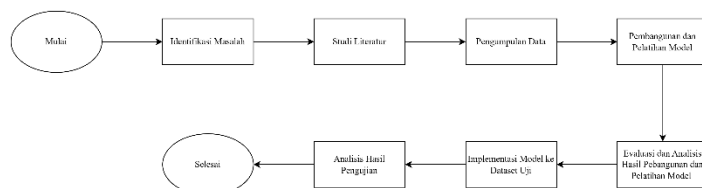
b. VGG16

- Phase 1: $lr = 0.001$ (1×10^{-3}), melatih *classification head* baru.
- Phase 2: $lr = 0.0001$ (1×10^{-4} , yaitu $LR / 10$), *fine-tuning* blok Conv5 dengan presisi tinggi.

Selain *learning rate* tetap, kedua model juga dilengkapi *callback ReduceLROnPlateau* yang secara adaptif mengurangi *learning rate* apabila *val_loss* tidak membaik selama beberapa *epoch* (*patience*=3, *factor*=0.1 untuk MobileNetV2; *factor*=0.5 untuk VGG16). Nilai minimum *learning rate* dibatasi pada $min_lr = 1 \times 10^{-7}$ (Phase 1) dan 1×10^{-8} (Phase 2) agar pelatihan tidak stagnan total.

Hasil *training* dan *testing* akan diperoleh setelah eksekusi program lengkap. Metrik evaluasi yang digunakan mencakup *Accuracy*, *Precision (macro)*, *Recall (macro)*, *F1-Score (macro)*, dan *MCC (Matthews Correlation Coefficient)*. Untuk MobileNetV2, performa terbaik diambil dari epoch dengan *val_accuracy* tertinggi sepanjang Phase 1 dan Phase 2 gabungan, disimpan secara otomatis via *ModelCheckpoint('mobilenetv2_best.h5')*. Untuk VGG16, model terbaik disimpan sebagai *best_vgg16_finetuned.keras*. Prediksi dilakukan pada data testing yang sepenuhnya terpisah (tidak pernah dilihat model selama pelatihan maupun validasi), kemudian dievaluasi menggunakan *classification_report* per kelas dan *confusion matrix* untuk analisis kesalahan prediksi antarspesies bunga.

3.5 Workflow atau Alur Penelitian



Gambar 3. 1 Alur Penelitian

1. Pengumpulan dataset.

Tahap pertama adalah pengumpulan gambar empat jenis bunga secara mandiri. Gambar dikumpulkan melalui dua cara, yaitu pengambilan foto langsung di lapangan menggunakan kamera *smartphone* dan pengunduhan dari sumber terbuka di internet. Setiap kelas dikumpulkan sebanyak 360 gambar sehingga total dataset berjumlah 1.440 gambar. Pada tahap ini juga dilakukan seleksi gambar secara manual untuk memastikan kualitas gambar memadai, tidak buram, dan objek bunga terlihat jelas. Variasi sudut pandang, pencahayaan, latar belakang, dan tahap mekar bunga sengaja dipertahankan untuk meningkatkan kemampuan generalisasi model.

2. *Preprocessing* data.

Tahap kedua adalah preprocessing yang bertujuan menyiapkan data agar sesuai dengan format masukan masing-masing model. *Preprocessing* yang dilakukan meliputi konversi format untuk seluruh gambar dikonversi ke ruang warna RGB, *resize* gambar diubah ukurannya menjadi 64×64 piksel untuk model berbasis Random Forest, dan 224×224 piksel untuk model berbasis *deep learning* (MobileNetV2 dan VGG16), normalisasi piksel untuk model Random Forest, nilai piksel dinormalisasi ke rentang [0, 1]. Untuk model MobileNetV2 dan VGG16, digunakan fungsi `preprocess_input()` yang menormalisasi nilai piksel ke rentang [-1, 1] sesuai standar pretrained ImageNet, ekstraksi fitur manual khusus untuk metode *Random Forest*, dilakukan ekstraksi fitur HOG (9 orientasi, sel 8×8, blok 2×2) dan fitur statistik warna RGB (*mean* dan standar deviasi tiap kanal), menghasilkan vektor fitur berdimensi 1.770, *encoding* label label kelas dikonversi menjadi bilangan bulat menggunakan *Label Encoder* untuk *Random Forest* dan SVM, serta *one-hot encoding* untuk model berbasis Keras, dan pembagian data, di mana dataset dibagi menggunakan *stratified split* dengan rasio 70% latih, 10% validasi, dan 20% uji [7].

3. Augmentasi Data

Tahap ketiga adalah augmentasi data yang diterapkan hanya pada data latih untuk memperbesar keberagaman data pelatihan tanpa pengumpulan data baru. Augmentasi mengikuti konfigurasi pada jurnal rujukan masing-masing yang meliputi refleksi horizontal, rotasi $\pm 45^\circ$, *zoom* 80–120%, serta translasi

horizontal dan vertikal. Augmentasi tidak diterapkan pada data validasi dan data uji agar evaluasi model mencerminkan kondisi data nyata tanpa modifikasi.

4. Pelatihan Model

Tahap keempat adalah pelatihan keenam model secara independen. *Random Forest* (RF) dilatih menggunakan fitur HOG sebagai masukan, dengan parameter 200 *estimator*, *max_features*='sqrt', dan *class_weight*='balanced', *Support Vector Machine* (SVM) dilatih menggunakan fitur HOG yang sama dengan kernel RBF, parameter C dan γ optimal ditentukan melalui *grid search* dengan 5-fold *cross-validation*, MobileNetV2 dilatih dengan dua fase, yaitu fase 1 (*feature extraction*, base frozen, lr=0,001) dan fase 2 (*fine-tuning*, 50 layer terakhir dibuka, lr=0,0001). *Optimizer* Adam dan *loss* categorical crossentropy, VGG16 dilatih dengan pendekatan serupa, yaitu fase 1 *base frozen*, fase 2 *fine-tuning* lapisan *classifier*, optimizer Adam, lr=0,00005, *batch size* 16, VGG16 + SVM, di mana VGG16 pretrained digunakan sebagai *feature extractor* yang menghasilkan vektor 4.096 dimensi dari lapisan fc4096, kemudian fitur tersebut digunakan untuk melatih SVM-RBF dengan strategi *One-vs-Rest* (OVR). Parameter C dan γ ditentukan melalui *grid search* 5-fold CV, dan terakhir MobileNetV2 + RF, di mana MobileNetV2 pretrained digunakan sebagai *feature extractor* yang menghasilkan vektor 1.280 dimensi dari lapisan GAP, kemudian fitur tersebut digunakan untuk melatih *Random Forest*.

5. Evaluasi model.

Tahap kelima adalah evaluasi kinerja seluruh model menggunakan data uji yang tidak pernah digunakan selama pelatihan. Metrik evaluasi yang digunakan meliputi *accuracy*, *precision*, *recall*, *F1-score*, *Matthews Correlation Coefficient* (MCC), dan *confusion matrix*. Selain itu, 5-fold *stratified cross-validation* diterapkan untuk memperoleh estimasi generalisasi yang lebih andal.

6. Perbandingan dan Analisis Hasil

Tahap keenam adalah perbandingan hasil evaluasi keenam model secara menyeluruh. Perbandingan dilakukan untuk mengidentifikasi model dengan performa terbaik dan menganalisis kelebihan serta kelemahan masing-masing pendekatan, yaitu ML konvensional, DL murni, dan hybrid CNN + klasifier.

7. Kesimpulan model terbaik.

Tahap terakhir adalah penarikan kesimpulan berdasarkan hasil perbandingan seluruh model. Kesimpulan mencakup metode terbaik untuk klasifikasi jenis bunga lokal Indonesia, peningkatan akurasi yang dicapai DL dibandingkan ML, serta rekomendasi pengembangan lebih lanjut.

3.6 Hasil dan Evaluasi Model

3.6.1 Model HOG + SVM

Pengujian model HOG + SVM dilakukan sebanyak tiga percobaan dengan variasi konfigurasi HOG dan pembagian data yang berbeda-beda untuk menemukan kombinasi terbaik. Setiap percobaan menggunakan GridSearchCV dengan *5-Fold Cross Validation* dan 32 kombinasi *hyperparameter* (160 fits total) untuk menemukan nilai C, gamma, dan kernel SVM yang optimal.

Tabel 3. 1 Variasi Konfigurasi HOG pada SVM

Percobaan	Konfigurasi HOG	Split Data	Jumlah Test
1	HOG (8×8)	70 / 15 / 15 → test = 432	432 gambar
2	HOG (16×16)	70 / 15 / 15 → test = 288	288 gambar
3	HOG (8×8, orient=12)	70 / 15 / 15 → test = 288	288 gambar

Pada percobaan 1, di mana menggunakan HOG (8×8) dan *split* data 70:15:15, *GridSearch* menemukan *hyperparameter* terbaik dengan kernel=rbf, C=10, gamma=0.01. Hasil evaluasi per kelas disajikan pada tabel berikut.

Tabel 3. 2 Percobaan 1 SVM+HOG

Kelas	Precision	Recall	F1-Score	Support
Melati Jepang	0,97	0,85	0,91	108
Tapak Dara	0,91	0,94	0,92	108
Bintaro	0,93	0,99	0,96	108
Melati Jakarta	0,88	0,91	0,89	108
Macro Average	0,92	0,92	0,92	432

Pada percobaan 2, dengan HOG (16×16) dan split data 70:15:15, GridSearch menemukan hyperparameter terbaik yang sama dengan Percobaan 1, yaitu kernel=rbf, C=10, gamma=0.01. Hasil evaluasi per kelas disajikan pada tabel berikut.

Tabel 3. 3 Percobaan 2 SVM+HOG

Kelas	Precision	Recall	F1-Score	Support
Melati Jepang	0,99	0,93	0,96	72
Tapak Dara	0,95	0,97	0,96	72
Bintaro	0,94	1,00	0,97	72
Melati Jakarta	0,97	0,93	0,95	72
Macro Average	0,96	0,96	0,96	288

Terakhir pada percobaan 3, dengan HOG (8×8, Orientasi=12) dan split data 70:15:15, GridSearch menemukan hyperparameter terbaik kernel=rbf, C=10, gamma=0.1. Berbeda dari dua percobaan sebelumnya pada nilai gamma yang lebih besar. Hasil evaluasi per kelas disajikan pada tabel berikut.

Tabel 3. 4 Percobaan 3 SVM+HOG

Kelas	Precision	Recall	F1-Score	Support
Melati Jepang	0,99	0,93	0,96	72
Tapak Dara	0,95	0,97	0,96	72
Bintaro	0,95	1,00	0,97	72
Melati Jakarta	0,97	0,94	0,96	72
Macro Average	0,96	0,96	0,96	288

Adapun hasil perbandingan pada ketiga percobaan tersebut sebagai berikut.

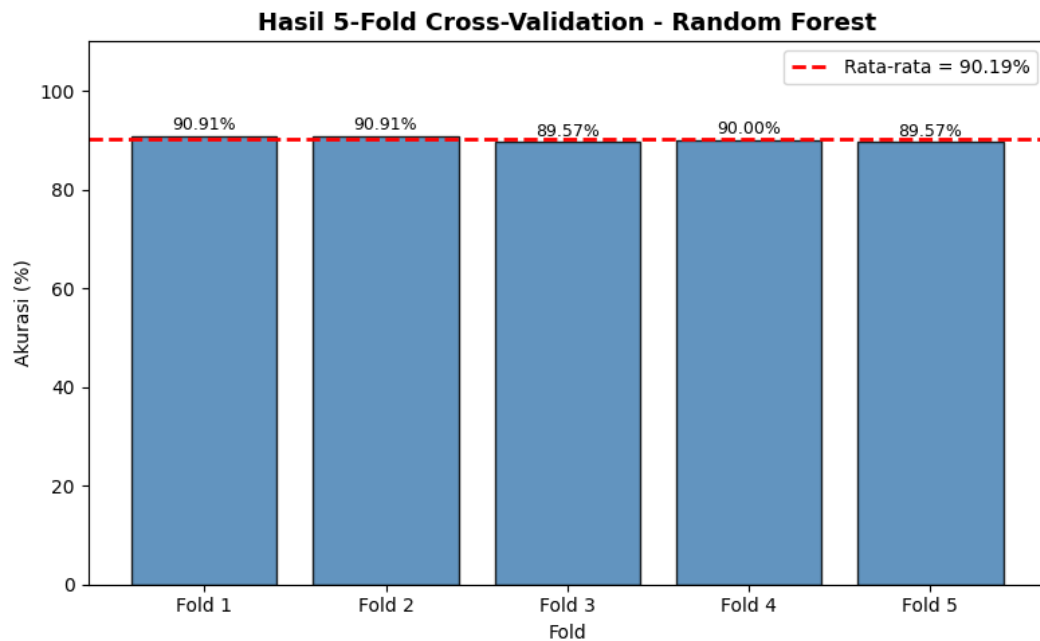
Tabel 3. 5 Perbandingan Percobaan SVM HOG

Percobaan	Konfigurasi	BestParams	Acc	Prec	Recall	F1-Score
1	HOG 8×8	C=10, $\gamma=0.1$, RBF	92,13%	0,9229	0,9213	0,9208
2	HOG 16×16	C=10, $\gamma=0.0$, RBF	95,83%	0,9593	0,9583	0,9582
3	HOG8×8 Orient=12	C=10, $\gamma=0.1$, RBF	96,18%	0,9625	0,9618	0,9617

Dari ketiga percobaan yang dilakukan, Percobaan 3 menghasilkan performa terbaik dengan akurasi 96,18%, diikuti Percobaan 2 dengan 95,83%, dan Percobaan 1 dengan akurasi terendah sebesar 92,13%. Perbedaan performa antara Percobaan 1 dan Percobaan 2 menunjukkan bahwa ukuran sel HOG berpengaruh signifikan terhadap kualitas fitur yang diekstrak. Sel berukuran 16×16 pada Percobaan 2 menghasilkan representasi fitur yang lebih ringkas dan generalisasi lebih baik dibandingkan sel 8×8 pada Percobaan 1, dengan peningkatan akurasi sebesar 3,70%. Percobaan 3 yang menggunakan konfigurasi HOG 8×8 dengan jumlah orientasi yang diperbanyak menjadi 12 (dari sebelumnya 9) terbukti menghasilkan performa terbaik dengan peningkatan akurasi sebesar 0,35% dibandingkan Percobaan 2. Penambahan jumlah orientasi memungkinkan HOG menangkap informasi arah gradien yang lebih detail dan beragam, sehingga fitur yang dihasilkan lebih diskriminatif dalam membedakan keempat kelas bunga. Hal ini juga berdampak pada perubahan gamma terbaik dari 0,01 menjadi 0,1, yang mengindikasikan bahwa ruang fitur dengan dimensi lebih kaya membutuhkan kernel RBF dengan jangkauan pengaruh yang lebih luas.

3. 6. 2 Model *Random Forest*

Sebelum evaluasi akhir pada data uji, dilakukan validasi silang (*cross-validation*) dengan metode 5-fold *stratified cross-validation* pada gabungan data latih dan data validasi. Metode ini membagi data menjadi lima lipatan (*fold*) secara bergantian sebagai data validasi dengan mempertahankan proporsi kelas yang seimbang pada setiap lipatan, sehingga memberikan estimasi kemampuan generalisasi model yang lebih andal dibandingkan evaluasi pada satu pembagian data tunggal.



Gambar 3. 2 Hasil 5-Fold Cross-Validation

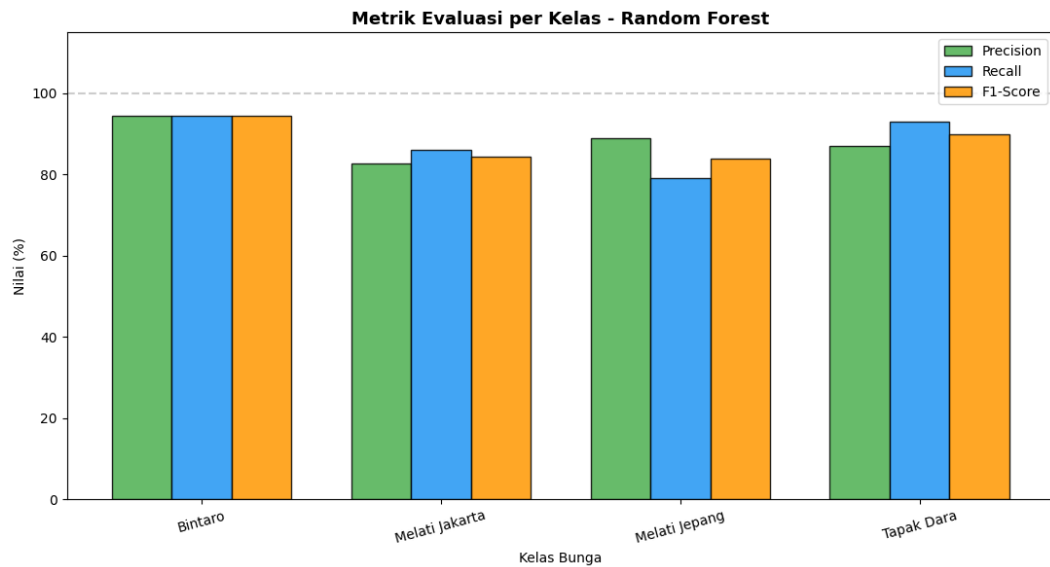
Tabel 3. 6 Hasil Cross-Validation

Fold	Akurasi
Fold 1	90,91%
Fold 2	90,91%
Fold 3	89.57%
Fold 4	90.00%
Fold 5	89.57%
Rata-rata	90.19%
Std. Deviasi	0.61%

Hasil *cross-validation* menunjukkan bahwa model *Random Forest* mampu mencapai akurasi rata-rata sebesar 90,19% dengan standar deviasi yang sangat kecil yaitu 0,61%. Nilai standar deviasi yang rendah ini mengindikasikan bahwa performa model bersifat konsisten dan stabil di seluruh lipatan data, sehingga model tidak mengalami *overfitting* terhadap subset data pelatihan tertentu. Konsistensi antar *fold* juga terlihat dari rentang nilai akurasi yang sempit, yaitu antara 89,57% hingga 90,91%, dengan selisih tertinggi hanya 1,34%. Hal ini menunjukkan bahwa pembagian data yang menggunakan strategi *stratified split* berhasil menjaga distribusi kelas yang representatif pada setiap *fold*.

Setelah proses pelatihan selesai, model *Random Forest* dievaluasi menggunakan data uji sebanyak 288 gambar (20% dari total dataset) yang tidak

pernah digunakan selama proses pelatihan maupun validasi. Hasil evaluasi secara keseluruhan disajikan pada tabel berikut.



Gambar 3. 3 Matrik Evaluasi per kelas RF

Tabel 3. 7 Matrik Evaluasi RF

Metrik	Nilai
Accuracy	88,19%
Precision	88,30%
Recall	88,19%
F1-Score	88,14%

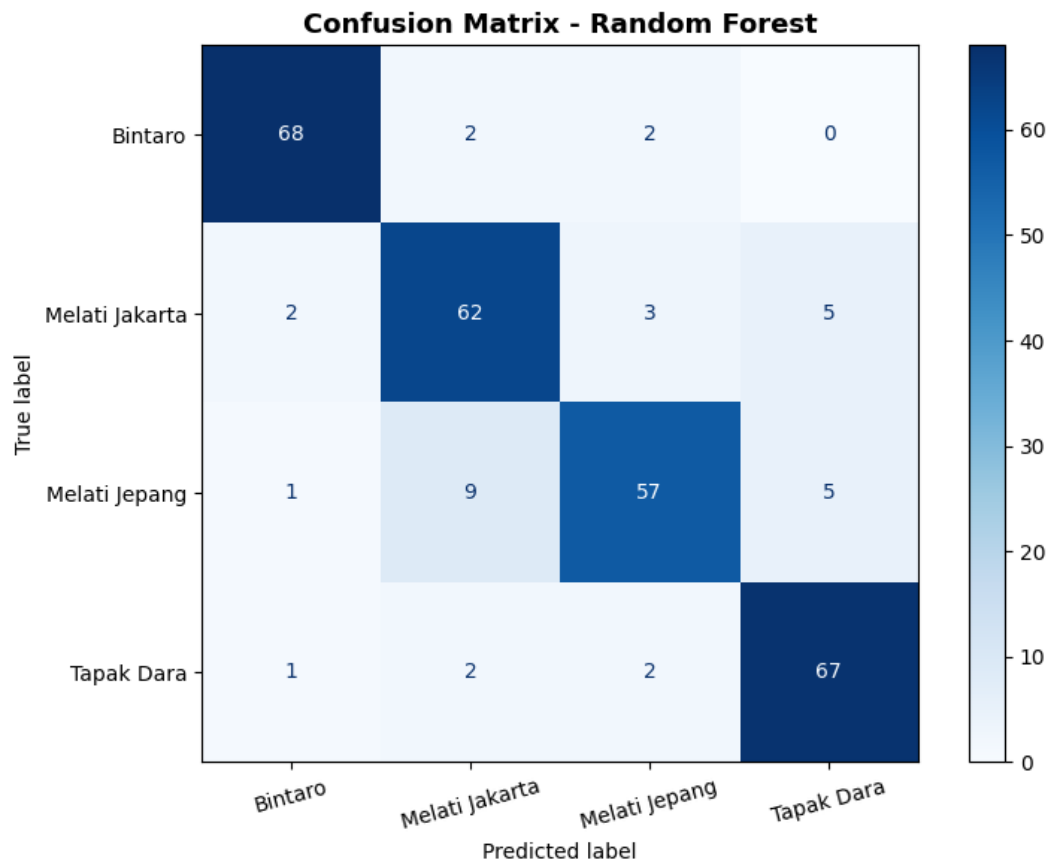
Model *Random Forest* berhasil mencapai akurasi sebesar 88,19% pada data uji. Nilai *precision* sebesar 88,30% menunjukkan bahwa dari seluruh prediksi positif yang dihasilkan model, 88,30% di antaranya merupakan prediksi yang benar. Nilai *recall* sebesar 88,19% menunjukkan bahwa model mampu mengenali 88,19% dari seluruh gambar bunga yang sebenarnya benar pada masing-masing kelas. Nilai *F1-score* sebesar 88,14% yang merupakan rata-rata harmonik antara *precision* dan *recall* mengkonfirmasi bahwa model memiliki keseimbangan yang baik antara ketepatan dan kelengkapan prediksi.

Terdapat selisih sebesar 2,00% antara akurasi *cross-validation* (90,19%) dan akurasi data uji (88,19%). Selisih ini tergolong wajar dan dalam batas yang dapat diterima, mengindikasikan bahwa model tidak mengalami *overfitting* yang signifikan dan mampu menggeneralisasi pola yang dipelajari dari data pelatihan ke data baru yang belum pernah dilihat sebelumnya.

Analisis lebih mendalam dilakukan terhadap performa model pada masing-masing kelas bunga. Hasil *classification report* disajikan pada tabel berikut.

Tabel 3. 8 Classification Report RF

Kelas	Precision	Recall	F1-Score	Support
Bintaro	0,94	0,94	0,94	72
Melati Jakarta	0,83	0,86	0,84	72
Melati Jepang	0,89	0,79	0,84	72
Tapak Dara	0,87	0,93	0,90	72
Macro avg	0,88	0,88	0,88	288
Weighted avg	0,88	0,88	0,88	288



Gambar 3. 4 Confusion Matrix RF

Secara keseluruhan, model *Random Forest* dengan fitur HOG dan statistik warna RGB menunjukkan performa yang cukup baik sebagai metode *Machine Learning baseline* pada tugas klasifikasi empat jenis bunga lokal Indonesia, dengan akurasi data uji sebesar 88,19% dan akurasi *cross-validation* sebesar 90,19%. Hasil ini

sejalan dengan temuan peneliti sebelumnya yang menunjukkan bahwa kombinasi fitur berbasis CNN dengan *classifier* konvensional mampu menghasilkan akurasi yang kompetitif, meskipun dalam penelitian ini fitur yang digunakan adalah HOG manual bukan fitur CNN.

3. 6. 3 Model VGG16 Fine-Tuning

Pada fase pertama, hanya lapisan *classifier head* yang dilatih selama maksimal 15 *epoch* dengan *learning rate* = 0,001. Mekanisme *early stopping* aktif memantau akurasi validasi dan menghentikan pelatihan secara otomatis apabila tidak ada peningkatan selama 5 *epoch* berturut-turut, serta memulihkan bobot terbaik (*best weights*) di akhir pelatihan.

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss
1	67,56%	1,0340	81,25%	0,5349
2	83,43%	0,4592	95,67%	0,1539
3	88,89%	0.3469	93,75%	0.1841
4	87,00%	0,4000	95,67%	0,1116
5	88,99%	0.3480	94,71%	0.1557
6	90,87%	0,2537	96,63%	0,0755
7	91,27	0.2823	96,63	0.0881
8	92,06%	0,2268	99,04%	0,0485
9	91,17%	0.2456	97,60%	0.0496
10	90,58%	0.2526	94,71%	0.1399
11	93,15%	0,2340	98,08%	0,0476
12	92,36%	0.2154	96,63%	0.0843
13	94,44%	0,2039	98,08%	0,0489

Early stopping aktif pada *epoch* ke-13 dan memulihkan bobot terbaik dari *epoch* ke-8 di mana akurasi validasi mencapai **99,04%** dengan *loss* validasi terendah sebesar 0,0485. Peningkatan akurasi validasi yang sangat pesat dari 81,25% pada *epoch* pertama menjadi 99,04% pada *epoch* ke-8 menunjukkan bahwa fitur-fitur yang telah dipelajari VGG16 dari dataset ImageNet sangat efektif ditransfer untuk tugas klasifikasi bunga, bahkan hanya dengan melatih *classifier head* baru.

Pada fase kedua, blok konvolusi terakhir VGG16 dibuka (*unfreeze*) sehingga total parameter yang dapat dilatih menjadi 7.870.212 parameter. Pelatihan

dilakukan dengan *learning rate* awal yang lebih kecil yaitu 0,0001 selama maksimal 10 *epoch*.

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss	LR
1	87,10%	0,4289	84,13%	0,8450	1e-4
3	93,45%	0,2020	81,73%	0,8882	1e-4
5	95,54%	0,1338	73,56%	1,1678	1e-4
6	96,53%	0,1028	100,00%	0,0070	5e-5
7	97,32%	0,0710	100,00%	0,0051	5e-5
8	98,21%	0,0640	100,00%	0,0015	5e-5
10	99,50%	0,0135	100,00%	0,0011	5e-5

Tabel 3. 9 Matriks Evlauasi VGG16

Metrik	Nilai
Accuracy	99,54%
Precision	0,9955 (99,55%)
Recall	0,9954 (99,54%)
Specificity	0,9985 (99,85%)
F1-Score	0,9954 (99,54%)
MCC	0,9939

Dinamika training fase kedua menunjukkan pola yang menarik. Pada *epoch* 1–5 dengan *learning rate* = 0,0001, akurasi validasi justru turun dari 84,13% menjadi 73,56% dan *loss* validasi meningkat drastis hingga 1,1678. Kondisi ini merupakan gejala *temporary degradation* yang umum terjadi pada awal *fine-tuning* ketika bobot lapisan konvolusi yang telah dioptimasi pada ImageNet mulai disesuaikan untuk domain baru, di mana bobot yang sebelumnya optimal belum sepenuhnya tersinkronisasi dengan *classifier head* yang baru [5]. Pada *epoch* ke-5, *ReduceLROnPlateau* menurunkan *learning rate* menjadi 5×10^{-5} . Penurunan *learning rate* ini berdampak sangat signifikan: akurasi validasi langsung melonjak dari 73,56% ke 100,00% pada *epoch* ke-6 dan bertahan hingga *epoch* ke-10. *Early stopping* memulihkan bobot terbaik dari *epoch* ke-6, yaitu titik di mana akurasi validasi pertama kali mencapai 100,00% dengan *loss* terendah sebesar 0,0070.

Model VGG16 yang telah dilatih dievaluasi pada data uji sebanyak 216 gambar. Model VGG16 berhasil mencapai akurasi yang sangat tinggi sebesar

99,54% pada data uji, merupakan peningkatan yang sangat signifikan sebesar 11,35% dibandingkan model *Random Forest* yang hanya mencapai 88,19%. Hasil ini mengkonfirmasi keunggulan *deep learning* berbasis *transfer learning* dibandingkan metode *machine learning* konvensional dalam tugas klasifikasi citra bunga, selaras dengan temuan Wu et al. (2023) [7] yang melaporkan bahwa VGG16 *standalone* mencapai akurasi 93,2% pada dataset gulma DeepWeeds — dan pada dataset bunga lokal ini VGG16 mampu melampaui angka tersebut secara substansial.

Nilai *precision* sebesar 99,55% menunjukkan bahwa hampir seluruh prediksi positif yang dihasilkan model merupakan prediksi yang benar, sementara nilai *recall* sebesar 99,54% mengindikasikan bahwa model berhasil mengenali hampir seluruh gambar bunga dari setiap kelas secara benar. Nilai *specificity* sebesar 99,85% yang mengukur kemampuan model menghindari prediksi *false positive* juga menunjukkan performa yang sangat baik. Nilai *F1-score* sebesar 99,54% yang hampir sempurna mencerminkan keseimbangan optimal antara *precision* dan *recall*. Nilai *Matthews Correlation Coefficient* (MCC) sebesar 0,9939 merupakan indikator yang sangat kuat karena MCC mempertimbangkan seluruh sel *confusion matrix* (TP, TN, FP, FN) secara bersamaan dan tidak bias terhadap distribusi kelas. Nilai MCC mendekati 1,0 menunjukkan bahwa model VGG16 memiliki kemampuan klasifikasi yang hampir sempurna pada seluruh kelas bunga yang diuji. Adapun analisis performa per kelas disajikan pada tabel berikut.

Tabel 3. 10 Analisis Performa VGG16 per Kelas

Kelas	Precision	Recall	F1-Score	Support
Melati Jakarta	0,98	1,00	0,99	54
Melati Jepang	1,00	0,98	0,99	54
Bintaro	1,00	1,00	1,00	54
Tapak Dara	1,00	1,00	1,00	54
Macro avg	1,00	1,00	1,00	216
Weighted avg	1,00	1,00	1,00	216

Hasil per kelas menunjukkan performa yang luar biasa di seluruh kelas. Bintaro dan Tapak Dara mencapai performa sempurna dengan *precision*, *recall*, dan

F1-score masing-masing sebesar 1,00 atau 100%. Model VGG16 tidak melakukan satu pun kesalahan klasifikasi pada kedua kelas ini dari total 108 gambar uji. Hasil ini mencerminkan kemampuan fitur hierarkis VGG16 dalam menangkap representasi visual yang sangat diskriminatif untuk kedua kelas tersebut.

Melati Jakarta mencapai *F1-score* sebesar 0,99 dengan *recall* sempurna 1,00 dan *precision* sedikit lebih rendah yaitu 0,98. Nilai *precision* 0,98 mengindikasikan bahwa terdapat satu gambar dari kelas lain yang salah diklasifikasikan sebagai Melati Jakarta (*false positive* = 1). Meskipun demikian, model berhasil mengidentifikasi seluruh 54 gambar Melati Jakarta pada data uji dengan benar (*false negative* = 0).

Melati Jepang mencapai *F1-score* serupa yaitu 0,99 dengan *precision* sempurna 1,00 dan *recall* 0,98. Nilai *recall* 0,98 mengindikasikan bahwa terdapat satu gambar Melati Jepang yang gagal dikenali dan salah diklasifikasikan ke kelas lain (*false negative* = 1). Perlu dicatat bahwa pada model *Random Forest*, pasangan Melati Jakarta–Melati Jepang merupakan sumber kesalahan terbesar dengan *recall* Melati Jepang hanya 79%. Dengan VGG16, masalah ini berhasil direduksi secara dramatis menjadi hanya satu kesalahan dari 54 gambar uji, membuktikan keunggulan fitur CNN dalam membedakan kelas-kelas dengan kemiripan visual tinggi.

3. 6. 4 Model MobileNetV2

Proses pelatihan arsitektur MobileNetV2 dilakukan menggunakan pendekatan *transfer learning* dua fase untuk mengoptimalkan adaptasi bobot pra-latih terhadap dataset gambar botani secara komputasional. Pada Phase 1, pembelajaran difokuskan secara eksklusif pada *classification head* dengan membekukan (*freeze*) lapisan dasar model. Fase ini mencatatkan konvergensi yang sangat cepat. Algoritma *early stopping* menghentikan proses komputasi pada *epoch* ke-14 dan melakukan restorasi bobot optimal dari *epoch* ke-9 yang meraih akurasi validasi puncak 100,00% dengan tingkat *loss* sebesar 0,0607. Pelatihan difokuskan pada *classification head* dengan *learning rate* awal 0,0010.

Tabel 3. 11 Phase 1 Pelatihan MobileNetV2

Epoch	Akurasi Latih	Loss Latih	Akurasi Validasi	Loss Validasi	Learning Rate
-------	------------------	---------------	---------------------	------------------	------------------

1	58,61%	1,0865	75,69%	0,6233	0,0010
2	71,88%	0,6123	77,78%	0,5946	0,0010
3	84,94%	0,4050	84,72%	0,3624	0,0010
4	81,25%	0,4149	84,72%	0,3552	0,0010
5	89,65%	0,3064	91,67%	0,2617	0,0010
6	87,50%	0,5034	92,36%	0,2561	0,0010
7	91,39%	0,2415	95,14%	0,1777	0,0010
8	87,50%	0,4588	95,83%	0,1790	0,0010
9	93,03%	0,1737	100,00%	0,0607	0,0010
10	90,62%	0,2538	99,31%	0,0613	0,0010
11	94,36%	0,1577	96,53%	0,1040	0,0010
12	96,88%	0,0911	96,53%	0,1053	0,0001
13	94,77%	0,1560	96,53%	0,0859	0,0001
14	93,75%	0,1098	96,53%	0,0854	0,0001

Pada phase 2, *unfreezing* pada lapisan blok konvolusional dilakukan dengan reduksi *learning rate* menjadi 0,0001 untuk memitigasi distorsi bobot secara drastis [2]. Fungsi *callback* ReduceLROnPlateau terpicu pada *epoch* ke-4, menurunkan *learning rate* ke tingkat 0,00001 guna mencegah osilasi gradien [1]. Kendati demikian, fase adaptasi lapisan dasar ini memicu indikasi *overfitting* minor pada data uji, sehingga *early stopping* memutuskan siklus pada *epoch* ke-6 dan merestorasi bobot dari *epoch* pertama Fase 2 yang memiliki akurasi validasi tertinggi di fase ini (96,53%) untuk menjaga tingkat stabilitas generalisasi model.

Tabel 3. 12 Phase 2 Pelatihan MobileNetV2

Epoch	Akurasi Latih	Loss Latih	Akurasi Validasi	Loss Validasi	Learning Rate
1	88,11%	0,3634	96,53%	0,0848	0,00010
2	93,75%	0,1986	96,53%	0,0852	0,00010
3	94,88%	0,1580	93,06%	0,1814	0,00010
4	96,88%	0,1693	92,36%	0,1887	0,00001
5	94,98%	0,1450	95,83%	0,1446	0,00001
6	100,00%	0,0395	95,83%	0,1444	0,00001

Model MobileNetV2 berhasil mencapai akurasi sebesar 97,22% pada data test. Nilai Tingginya nilai metrik *Matthews Correlation Coefficient* (MCC) yang menyentuh angka 0,9631 secara empiris mengonfirmasi bahwa ketepatan prediksi klasifikasi sangat tangguh (*robust*). Metrik ini memastikan bahwa tingkat akurasi didistribusikan secara proporsional pada kuadran konfusi matriks tanpa adanya bias prediksi atau *random guessing* terhadap kelas tertentu.

Tabel 3. 13 Metrik Evaluasi MobileNetV2

Metrik	Nilai
Accuracy	97.22%
Precision	0.9730
Recall	0.9722
F1-Score	0.9724
MCC	0.9631

Evaluasi performa tingkat kelas (*class-level performance*) pada arsitektur MobileNetV2 mendemonstrasikan variasi kapabilitas fungsi aktivasi model dalam mendiskriminasi fitur morfologis spesifik antar-spesies. Pengujian pada 288 sampel observasi yang berdistribusi secara ekuivalen (*balanced dataset* dengan 72 sampel per kelas) memetakan dinamika pengenalan fitur visual.

Tabel 3. 14 Evaluasi Performa MobileNetV2 per kelas

Kelas Bunga	Precision	Recall	F1-Score	Support (Sampel)
Melati Jakarta	0,95	0,97	0,96	72
Melati Jepang	1,00	0,96	0,98	72
Bintaro	1,00	0,99	0,99	72
Tapak Dara	0,95	0,97	0,96	72
Macro Average	0,97	0,97	0,97	288

3. 6. 5 Model Hybrid VGG16 + SVM

Pelatihan VGG16 dilakukan dalam dua fase. Pada Phase 1 (Head Training), seluruh layer backbone VGG16 dibekukan dan hanya classification head yang

dilatih dengan learning rate 0,001 selama maksimal 15 epoch. Hasil pelatihan Phase 1 per epoch disajikan pada tabel berikut.

Tabel 3. 15 Phase 1 Pelatihan Model VGG16

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss	Learning Rate
1	63,42%	1,0522	67,97%	1,1097	0,0010
2	84,38%	0,5044	69,53%	1,0845	0,0010
3	84,22%	0,4556	94,53%	0,1666	0,0010
4	90,62%	0,2823	94,53%	0,1607	0,0010
5	88,63%	0,3337	94,53%	0,2181	0,0010
6	93,75%	0,1249	94,53%	0,2123	0,0010
7	92,11%	0,2242	95,31%	0,0954	0,0010
8	93,75%	0,1216	95,31%	0,0912	0,0010
9	91,60%	0,2611	97,66%	0,0580	0,0010
10	84,38%	0,5229	97,66%	0,0676	0,0010
11	93,24%	0,1945	98,44%	0,0487	0,0010
12	90,62%	0,1613	98,44%	0,0479	0,0010
13	94,67%	0,1803	98,44%	0,0409	0,0010
14	96,88%	0,1139	98,44%	0,0396	0,0010
15	93,95%	0,1805	97,66%	0,0551	0,0010

Training Phase 1 berjalan penuh hingga epoch ke-15 tanpa dipotong *early stopping*, yang mengindikasikan model masih terus belajar sepanjang phase ini. Bobot model dikembalikan ke epoch ke-11 yang menghasilkan val_accuracy tertinggi sebesar 98,44% dengan val_loss sebesar 0,0487. Tren pelatihan menunjukkan dua pola menarik. Pertama, terdapat lonjakan val_accuracy yang signifikan dari 69,53% pada epoch ke-2 menjadi 94,53% pada epoch ke-3, menandakan model berhasil menemukan representasi fitur yang tepat secara tiba-tiba. Kedua, pada epoch ke-10 terjadi penurunan sementara pada train_accuracy menjadi 84,38% namun val_accuracy tetap stabil di 97,66%, yang menunjukkan model sedang dalam proses generalisasi yang baik dan tidak hanya menghafal data training.

Pada Phase 2 (Fine-Tuning), blok conv5 VGG16 (4 layer terakhir backbone) dibuka untuk dilatih ulang dengan learning rate diturunkan menjadi 0,0001. Hasil pelatihan Phase 2 disajikan pada tabel berikut.

Tabel 3. 16 Phase 2 Pelatihan VGG16

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss	Learning Rate
1	89,55%	0,3357	89,84%	0,3658	0,00010
2	93,75%	0,2713	90,62%	0,2927	0,00010
3	93,65%	0,1799	97,66%	0,0588	0,00010
4	100,00%	0,0200	96,88%	0,0677	0,00010
5	95,90%	0,1029	90,62%	0,3860	0,00010

6	90,62%	0,2406	93,75%	0,2580	0,00010
7	97,54%	0,0671	100,00%	0,0041	0,00005
8	100,00%	0,0028	100,00%	0,0036	0,00005
9	98,67%	0,0442	98,44%	0,0167	0,00005
10	93,75%	0,2318	99,22%	0,0127	0,00005

Phase 2 menyelesaikan seluruh 10 epoch yang dijadwalkan. Pada epoch ke-4 terjadi *train accuracy* sempurna 100% namun diikuti penurunan *val_accuracy* pada epoch ke-5 menjadi 90,62% dan *val_loss* yang melonjak menjadi 0,3860, yang mengindikasikan gejala *overfitting* sementara. Pada epoch ke-6, *ReduceLROnPlateau* secara otomatis menurunkan learning rate dari 0,0001 menjadi 0,00005 karena *val_loss* tidak membaik selama 3 epoch. Penurunan learning rate ini terbukti sangat efektif, di mana pada epoch ke-7 terjadi lompatan performa yang dramatis di mana *val_accuracy* melonjak dari 93,75% menjadi 100,00% dan *val_loss* turun drastis dari 0,2580 menjadi 0,0041. Bobot terbaik dipilih dari epoch ke-7 sebagai titik pertama tercapainya akurasi validasi sempurna, dan training dinyatakan selesai setelah epoch ke-10.

Sebelum melatih SVM, fitur diekstrak terlebih dahulu dari layer *Global Average Pooling* (GAP) VGG16 yang telah dilatih sebelumnya. Proses ekstraksi dilakukan terhadap seluruh 1.440 gambar dataset dan menghasilkan vektor fitur berdimensi 512 per gambar, sehingga total matriks fitur berukuran (1.440×512) . Fitur kemudian dibagi menjadi data training sebanyak 1.152 gambar dan data test sebanyak 288 gambar sesuai split 80/20. Proses ekstraksi berlangsung selama 8 detik dengan kecepatan 156 ms/step.

Adapun empat jenis kernel SVM diuji secara sistematis menggunakan fitur lengkap 512 dimensi tanpa seleksi fitur. Hasil perbandingan keempat kernel disajikan pada Tabel 1 berikut.

Tabel 3. 17 Perbandingan kernel

Kernel	Accuracy	Precision	Recall	F1-Score
Linear	98,26%	0,9832	0,9826	0,9825
RBF	97,92%	0,9793	0,9792	0,9791
Sigmoid	88,89%	0,8910	0,8889	0,8891
Poly	82,64%	0,8698	0,8264	0,8155

Dari keempat kernel yang diuji, kernel Linear menghasilkan performa terbaik dengan akurasi 98,26% diikuti kernel RBF dengan akurasi 97,92%. Selisih

keduanya hanya 0,34% yang menunjukkan bahwa fitur VGG16 berdimensi 512 memiliki sifat yang hampir *linearly separable* di ruang fitur tersebut, sehingga kernel Linear yang lebih sederhana sudah cukup untuk memisahkan keempat kelas bunga secara efektif. Kernel Sigmoid dan Poly menunjukkan performa yang jauh lebih rendah, masing-masing 88,89% dan 82,64%, mengindikasikan bahwa kedua kernel tersebut kurang cocok untuk karakteristik distribusi fitur deep learning pada dataset ini. Kernel Linear ditetapkan sebagai model terbaik dan dievaluasi secara lebih rinci. Hasil prediksi per kelas disajikan pada tabel berikut.

Tabel 3. 18 Hasil Prediksi Model VGG16 + SVM per Kelas

Kelas	Benar Diklasifikasi	Salah Diklasifikasi	Total
Melati Jakarta	72	0	72
Melati Jepang	67	5 (3 → Mel. Jakarta, 2 → Tapak Dara)	72
Bintaro	72	0	72
Tapak Dara	72	0	72
Total	283	5	288

Berdasarkan *confusion matrix*, seluruh kesalahan klasifikasi terpusat pada kelas Melati Jepang dengan 5 gambar salah diklasifikasikan, 3 gambar diprediksi sebagai Melati Jakarta, dan 2 gambar diprediksi sebagai Tapak Dara. Sementara itu, kelas Melati Jakarta, Bintaro, dan Tapak Dara masing-masing mencapai klasifikasi sempurna dengan 72 dari 72 gambar diprediksi dengan benar.

Model VGG16 + SVM dengan kernel Linear berhasil mencapai akurasi 98,26% yang menunjukkan efektivitas pendekatan *hybrid* antara *deep learning* sebagai *feature extractor* dan SVM sebagai *classifier*. Pendekatan ini sejalan dengan pernyataan Fei et al. (2023) yang menyebutkan bahwa kombinasi CNN sebagai *feature extractor* dengan SVM sebagai *classifier* mampu menghasilkan akurasi klasifikasi hingga 99,8%. Kesalahan yang terpusat pada kelas Melati Jepang mengindikasikan bahwa fitur visual bunga ini memiliki kemiripan tertentu dengan Melati Jakarta maupun Tapak Dara dalam ruang fitur 512 dimensi VGG16. Hal ini berbeda dengan model VGG16 Fine-Tuning (*end-to-end*) yang hanya menghasilkan 1 kesalahan, menunjukkan bahwa proses *fine-tuning* memungkinkan model mengoptimalkan representasi fitur secara lebih spesifik untuk membedakan kelas-kelas yang visual serupa dibandingkan menggunakan fitur GAP yang bersifat lebih

umum. Pemilihan kernel Linear sebagai yang terbaik juga memberikan keuntungan praktis berupa waktu inferensi yang lebih cepat dan interpretasi model yang lebih sederhana dibandingkan kernel RBF, sehingga lebih sesuai untuk implementasi pada sistem klasifikasi bunga secara *real-time*.

3. 6. 6 Model Hybrid MobileNetV2 + *Random Forest*

Pelatihan model MobileNetV2 + *Random Forest* menggunakan MobileNetV2 sebagai feature extractor yang dilatih dalam dua fase. Pada Phase 1 (Head Training), seluruh layer backbone dibekukan dan hanya classification head yang dilatih dengan learning rate 0,001 selama maksimal 20 epoch. Hasil pelatihan Phase 1 per epoch disajikan pada tabel berikut.

Tabel 3. 19 Phase 1 Pelatihan MobileNetV2

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss	Learning Rate
1	54,30%	1,2271	69,44%	0,7919	0,0010
2	75,00%	0,5888	68,75%	0,7862	0,0010
3	80,94%	0,4971	93,06%	0,2808	0,0010
4	87,50%	0,3790	93,06%	0,2715	0,0010
5	88,93%	0,3290	94,44%	0,2019	0,0010
6	93,75%	0,1766	95,14%	0,1943	0,0010
7	91,50%	0,2166	97,22%	0,1209	0,0010
8	90,62%	0,4026	96,53%	0,1163	0,0010
9*	92,62%	0,1996	99,31%	0,0632	0,0010
10	100,00%	0,0421	99,31%	0,0626	0,0010
11	95,08%	0,1489	97,22%	0,0626	0,0010
12	90,62%	0,5913	97,22%	0,0633	0,0010
13	94,35%	0,1708	98,61%	0,0449	0,0010
14	93,75%	0,1991	98,61%	0,0468	0,0010

Training Phase 1 berhenti pada epoch ke-14 melalui mekanisme *early stopping* karena val_accuracy tidak meningkat selama 5 epoch berturut-turut sejak epoch ke-9. Bobot model kemudian dikembalikan ke epoch ke-9 yang

menghasilkan *val_accuracy* tertinggi sebesar 99,31% dengan *val_loss* sebesar 0,0632. Tren pelatihan menunjukkan peningkatan yang stabil sejak epoch pertama (54,30%) hingga mencapai konvergensi di sekitar epoch ke-9 hingga ke-10. Perlu dicatat bahwa pada epoch ke-10 terjadi *train accuracy* sempurna sebesar 100% namun *val_accuracy* tidak meningkat, yang mengindikasikan model mulai menghafal data training sehingga *early stopping* bekerja dengan tepat.

Pada Phase 2 (Fine-Tuning), sebanyak 50 layer terakhir backbone dibuka untuk dilatih ulang dengan learning rate yang diturunkan menjadi 0,0001. Hasil pelatihan Phase 2 disajikan pada tabel berikut.

Tabel 3. 20 Phase 2 pelatihan MobileNetV2

Epoch	Train Accuracy	Train Loss	Val Accuracy	Val Loss	Learning Rate
1	87,91%	0,3576	97,92%	0,0595	0,00010
2	84,38%	0,2730	97,22%	0,0619	0,00010
3	94,57%	0,1424	95,14%	0,1511	0,00010
4	93,75%	0,1713	95,14%	0,1526	0,00001
5	96,31%	0,1162	95,14%	0,1429	0,00001

Pada epoch ke-3 Phase 2, *ReduceLROnPlateau* menurunkan *learning rate* dari 0,0001 menjadi 0,00001 karena *val_loss* tidak membaik. Meskipun demikian, performa validasi tetap tidak pulih dan training berhenti pada epoch ke-5 melalui *early stopping*. Bobot model dikembalikan ke epoch ke-1 Phase 2 dengan *val_accuracy* terbaik sebesar 97,92%. Kondisi ini menunjukkan bahwa membuka 50 layer backbone justru menyebabkan *overfitting* pada dataset berukuran kecil, sehingga bobot Phase 1 yang lebih stabil lebih optimal untuk dipertahankan. Fitur yang diekstrak dari bobot terbaik ini kemudian digunakan sebagai input untuk melatih *classifier Random Forest*. Setelah proses ekstraksi fitur dari *backbone* selesai, *classifier Random Forest* dilatih menggunakan fitur tersebut dan dievaluasi pada data test yang sepenuhnya terpisah. Hasil evaluasi keseluruhan disajikan pada tabel berikut.

Tabel 3. 21 Metrik Evaluasi MobileNetV2 + RF

Metrik	Nilai
--------	-------

Accuracy	99,31%
Precision	0,9932
Recall	0,9931
F1-Score	0,9931
MCC	0,9908

Tabel 3. 22 Hasil Prediksi Model MobileNetV2 + Random Forest per Kelas

Kelas	Benar Diklasifikasi	Salah Diklasifikasi	Total
Melati Jakarta	71	1 (→ Melati Jepang)	72
Melati Jepang	72	0	72
Bintaro	72	0	72
Tapak Dara	71	1 (→ Melati Jepang)	72
Total	286	2	288

Model MobileNetV2 + *Random Forest* berhasil mencapai akurasi 99,31% pada data test dengan hanya 2 dari 288 gambar yang salah diklasifikasikan. Nilai *precision* 0,9932 dan *recall* 0,9931 yang hampir identik menunjukkan model tidak condong ke arah *false positive* maupun *false negative*. *F1-Score* sebesar 0,9931 mengonfirmasi keseimbangan yang sangat baik antara keduanya.

Berdasarkan confusion matrix, seluruh kesalahan klasifikasi terjadi pada kelas Melati Jakarta dan Tapak Dara yang masing-masing satu gambar salah diprediksi sebagai Melati Jepang. Kelas Melati Jepang dan Bintaro mencapai klasifikasi sempurna dengan 72 dari 72 gambar berhasil diprediksi dengan benar. Kesalahan pada Melati Jakarta dan Tapak Dara kemungkinan disebabkan oleh kemiripan visual tertentu dengan Melati Jepang dalam kondisi pencahayaan atau sudut pengambilan gambar yang spesifik.

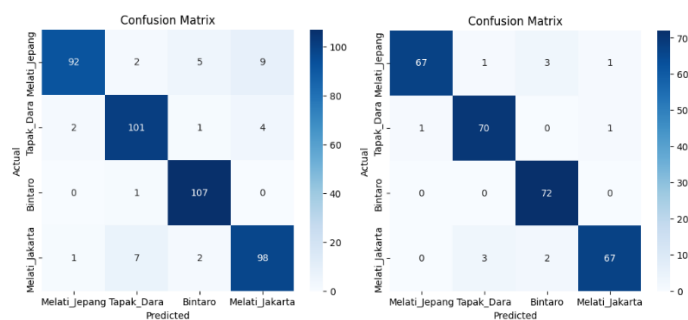
Nilai *MCC* sebesar 0,9908 yang sangat mendekati nilai sempurna 1,0 mengonfirmasi bahwa performa model sangat konsisten di seluruh kelas. Pendekatan MobileNetV2 + *Random Forest* terbukti efektif dengan memanfaatkan kemampuan MobileNetV2 sebagai *feature extractor* yang kuat dikombinasikan

dengan kemampuan *Random Forest* dalam membangun batas keputusan yang robust melalui mekanisme *ensemble* dari banyak *decision tree*.

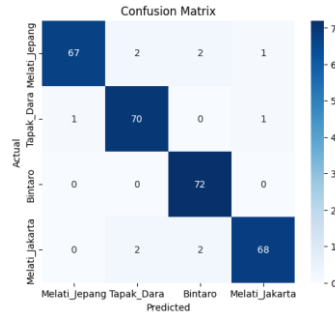
3.7 Visualisasi Hasil

Subbab ini menyajikan seluruh visualisasi yang dihasilkan dari proses pelatihan, evaluasi, dan prediksi keempat model yang diuji dalam penelitian ini, yaitu HOG + SVM, *Random Forest*, VGG16 Fine-Tuning, MobileNetV2, VGG16 + SVM, dan MobileNetV2 + *Random Forest*.

3.7.1 Model HOG + SVM



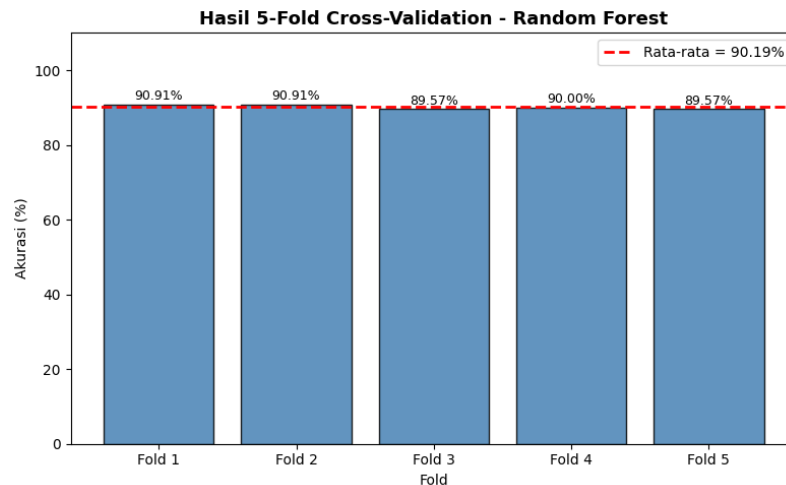
Gambar 3. 5 Confusion matrix HOG + SVM



Evaluasi pada ketiga visualisasi matriks konfusi mendemonstrasikan kinerja arsitektur *machine learning* yang mengandalkan *Histogram of Oriented Gradients* (HOG) sebagai ekstraktor fitur dan *Support Vector Machine* (SVM) sebagai pengklasifikasi akhir. Deskriptor HOG secara eksklusif memetakan distribusi intensitas gradien lokal dan arah tepi (*edge orientations*) pada jaringan sel matriks gambar. Pada percobaan 1, model dievaluasi menggunakan 432 sampel pengujian. Model mencatatkan tingkat kesalahan klasifikasi silang (*cross-classification*) tertinggi. Batas keputusan SVM terlihat paling tidak stabil pada kelas Melati Jepang, di mana 16 sampel terdistribusi keliru ke tiga kelas lainnya dan Melati Jakarta dengan 10 sampel salah prediksi. Pada percobaan 2 dan 3, dievaluasi

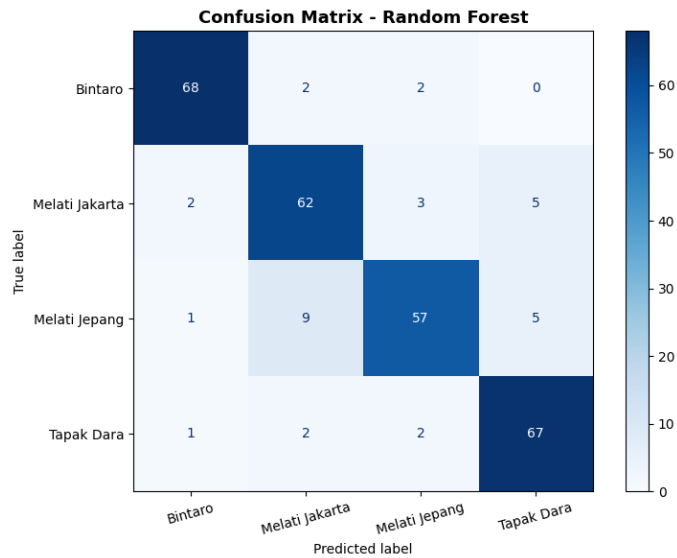
menggunakan 288 sampel pengujian. Kedua percobaan ini menunjukkan pola matriks yang hampir identik dan lebih stabil dibandingkan Percobaan 1. Secara spesifik, model mencapai pemisahan absolut (*recall* 100%) pada kelas Bintaro di mana 72 sampel diklasifikasikan secara mutlak tanpa memicu kondisi *False Negative* maupun *False Positive*.

3. 7. 2 Model Random Forest



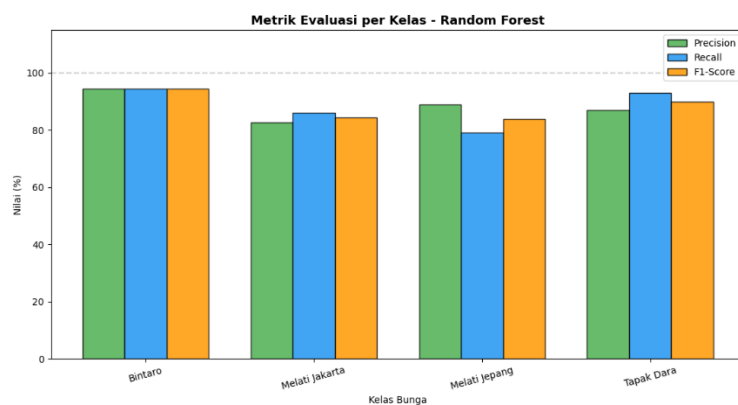
Gambar 3. 6 perbandingan Cross-Validation

Visualisasi grafik batang tersebut menampilkan hasil evaluasi stabilitas dan kemampuan generalisasi model *Random Forest* menggunakan metode *5-Fold Cross-Validation*. Pendekatan divalidasi silang ini membagi dataset ke dalam 5 partisi (*folds*) yang berukuran sama, di mana setiap iterasi menggunakan 4 partisi untuk pelatihan dan 1 partisi untuk pengujian secara bergantian, sehingga meminimalisir bias partisi data. Model mencapai akurasi rata-rata global sebesar 90,19%, yang direpresentasikan oleh garis putus-putus merah horizontal. Angka ini menegaskan bahwa algoritma *ensemble learning* berbasis pohon keputusan (*decision trees*) ini beroperasi dengan sangat baik pada data citra yang diujikan. Nilai tertinggi dicapai pada Fold 1 dan Fold 2 (90,91%), sementara nilai terendah berada pada Fold 3 dan Fold 5 (89,57%). Rentang varians yang sangat sempit ini secara definitif mengonfirmasi ketiadaan indikasi *overfitting* maupun *underfitting*. Model berhasil mempelajari pola fitur data secara representatif, bukan sekadar menghafal subset data pelatihan tertentu.

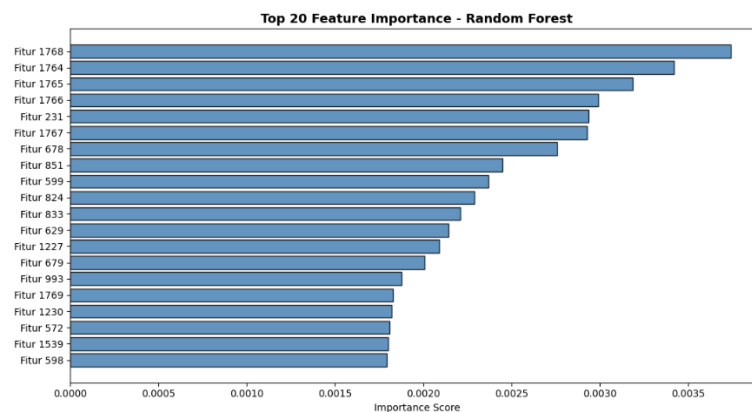


Gambar 3. 7 Confusion Matrix Random Forest

Visualisasi *confusion matrix* di atas menggambarkan hasil pengujian algoritma *Random Forest* tanpa bantuan metode ekstraksi fitur spasial (*deep learning*). Pengujian dilakukan pada dataset yang seimbang (*balanced dataset*), terdiri dari 288 observasi dengan alokasi 72 sampel untuk masing-masing kelas. Secara keseluruhan, model berhasil mengklasifikasikan 254 sampel secara tepat, menghasilkan tingkat akurasi global sebesar 88,19%. Model mencapai tingkat sensitivitas (*recall*) terbaik pada kelas Bintaro dengan keberhasilan memprediksi 68 dari 72 sampel aktual. Kelas Tapak Dara menyusul dengan 67 prediksi benar. Penurunan performa paling tajam terjadi pada kelas Melati Jepang, di mana model hanya mampu mengenali 57 dari 72 sampel aktual. Sisanya mengalami *misclassification* yang terdistribusi ke kelas Melati Jakarta (9 sampel), Tapak Dara (5 sampel), dan Bintaro (1 sampel).



Visualisasi grafik batang di atas menggambarkan kinerja algoritma *Random Forest* berdasarkan tiga metrik evaluasi utama *Precision*, *Recall*, dan *F1-Score* untuk setiap kelas observasi. Model mencatatkan performa paling stabil dan superior secara eksklusif pada kelas Bintaro. Ketiga metrik berada pada ekuilibrium yang ekuivalen dan konsisten menyentuh rentang tertinggi mendekati 95%. Hal ini membuktikan bahwa Bintaro memiliki representasi fitur visual yang dominan diskriminatif, sehingga batas partisi *node* pada pohon keputusan dapat memisahkan kelas ini dengan sangat presisi. Kesulitan terbesar model terpusat pada pemisahan kelas Melati Jepang dan Melati Jakarta. Terjadi fenomena *trade-off* (tarik-ulur) metrik yang tajam di antara keduanya.

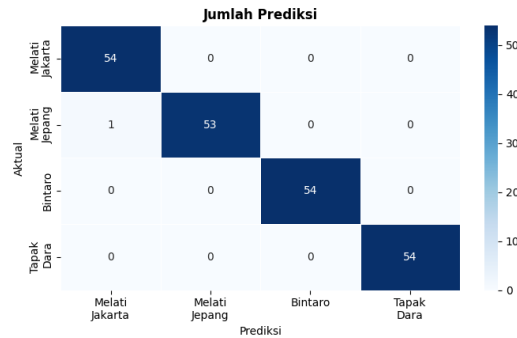


Gambar 3. 8 Feature Importance Random Forest

Visualisasi grafik batang *Top 20 Feature Importance* mengilustrasikan distribusi bobot kontribusi dari 20 fitur paling diskriminatif yang digunakan oleh algoritma *Random Forest* dalam menentukan partisi batas keputusan (*decision boundary*). Secara fundamental, metrik ini dihitung berdasarkan *Mean Decrease in Impurity* (Gini Importance), yang mengukur seberapa efektif sebuah fitur dalam mereduksi ketidakpastian (entropi) saat memisahkan kelas-kelas observasi pada setiap *node* pohon keputusan. Karakteristik paling mencolok dari grafik ini adalah rentang nilai *Importance Score* yang sangat kecil secara absolut. Fitur dengan peringkat tertinggi (Fitur 1768) hanya memiliki bobot sekitar 0,0037. Hal ini membuktikan bahwa model beroperasi pada ruang vektor berdimensi sangat tinggi (*high-dimensional feature space*). Karena total probabilitas kepentingan fitur harus berjumlah 1,0 (100%), distribusi bobot yang sangat merata dan rendah ini

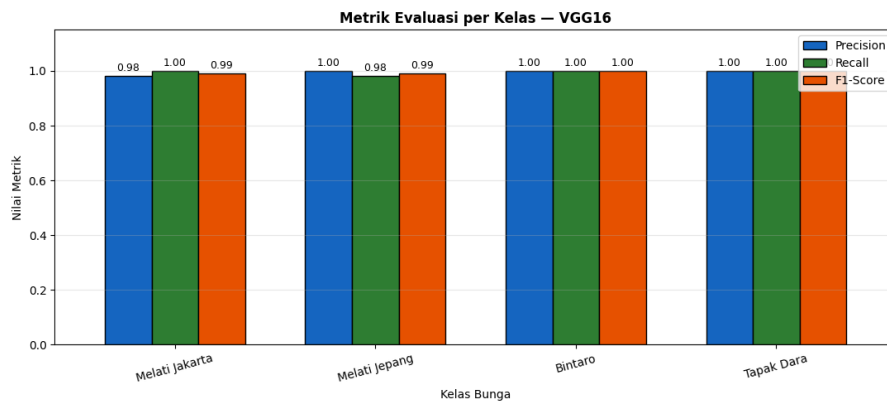
mengindikasikan bahwa kapasitas prediktif model tidak bergantung pada segelintir fitur emas (*golden features*), melainkan merupakan hasil agregasi kolektif dari ribuan fitur deterministik yang tergolong lemah (*weak learners*)

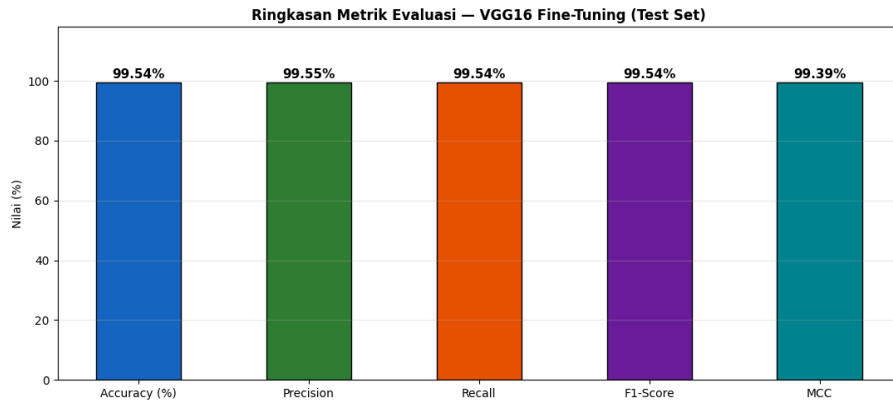
3. 7. 3 Model VGG16 Fine-Tuning



Gambar 3. 9 Confusion matrix VGG16

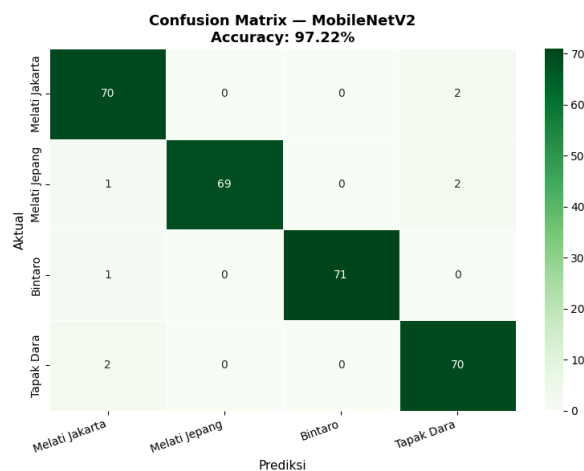
Visualisasi confusion matrix tersebut merangkum hasil evaluasi distribusi prediksi aktual, yang secara metrik merepresentasikan performa pengujian model (*fine-tuning*) pada 216 sampel data uji. Dataset dievaluasi dalam kondisi yang sepenuhnya seimbang (*balanced dataset*), yakni mengalokasikan tepat 54 sampel observasi untuk masing-masing kelas. Dari gambar, model berhasil mengenali seluruh representasi aktual pada kelas Melati Jakarta, Bintaro, dan Tapak Dara. Sebanyak 54 sampel pada masing-masing kelas tersebut diidentifikasi secara tepat sasaran tanpa adanya kasus *False Negative*. Satu-satunya penyimpangan klasifikasi pada keseluruhan ruang pengujian terjadi pada kelas Melati Jepang. Dari 54 sampel aktual, terdapat 1 sampel yang disalahartikan sebagai Melati Jakarta.





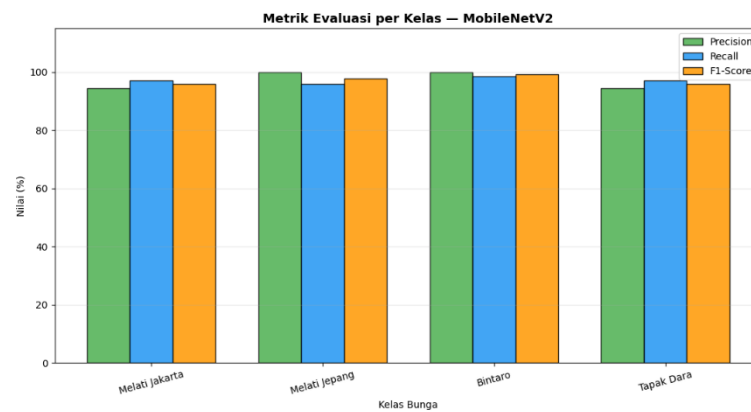
Visualisasi ringkasan metrik evaluasi mendemonstrasikan performa model VGG16 setelah melalui tahap *fine-tuning*. Secara keseluruhan, model mencapai tingkat kinerja yang sangat baik dengan Akurasi (99,54%), Presisi (99,55%), Recall (99,54%), dan F1-Score (99,54%). Keseimbangan skor di antara keempat parameter dasar ini mengindikasikan bahwa model tidak mengalami bias terhadap kelas mayoritas atau minoritas, dan beroperasi secara optimal pada distribusi data yang seimbang (*balanced dataset*). Pencapaian yang paling signifikan adalah nilai MCC (*Matthews Correlation Coefficient*) yang menyentuh angka 99,39%. Dalam evaluasi klasifikasi multikelas, MCC dianggap sebagai metrik statistik yang paling komprehensif karena mengevaluasi seluruh kuadran *confusion matrix* (*True Positives, True Negatives, False Positives, False Negatives*). Skor MCC yang mendekati 100% ini secara empiris membuktikan bahwa prediksi model sangat solid dan sama sekali bukan merupakan hasil probabilitas acak (*random guessing*).

3. 7. 4 MobileNetV2

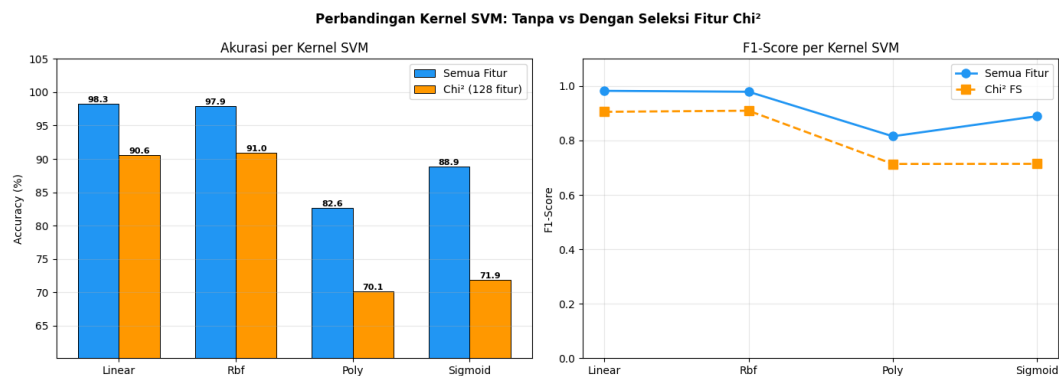


Gambar 3. 10 Confusion Matrix MobileNetV2

Visualisasi merepresentasikan confusion matrix, di mana model berhasil memprediksi 280 sampel secara presisi dari total 288 observasi yang terdistribusi secara seimbang (*balanced dataset* dengan 72 sampel per kelas). Model mencapai tingkat pengenalan paling tinggi pada representasi kelas Bintaro di mana 71 dari 72 sampel diidentifikasi secara tepat sasaran, dan hanya 1 sampel yang mengalami kesalahan prediksi sebagai Melati Jakarta. Penurunan performa paling signifikan terisolasi pada kelas Melati Jepang. Dari 72 observasi aktual, terdapat 3 sampel yang gagal dikenali, 1 diprediksi keliru sebagai Melati Jakarta, dan 2 diprediksi sebagai Tapak Dara. Matriks menunjukkan adanya kesalahan yang simetris, yaitu 2 sampel aktual Melati Jakarta disalahartikan sebagai Tapak Dara, dan sebaliknya, 2 sampel aktual Tapak Dara disalahartikan sebagai Melati Jakarta.

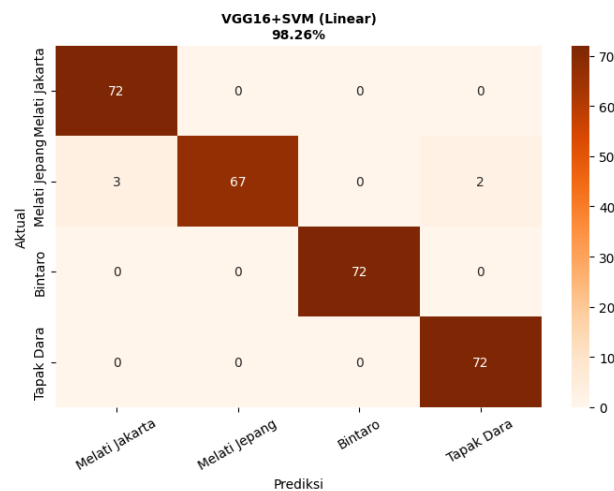


3. 7. 5 Model SVM + VGG16



Visualisasi menggambarkan evaluasi komparatif performa SVM menguji empat variasi fungsi kernel, yaitu Linear, RBF, Polinomial, dan Sigmoid dalam dua skenario dimensi fitur, yang menggunakan seluruh representasi spasial dibandingkan dengan reduksi dimensi menjadi 128 fitur melalui seleksi statistik. Grafik akurasi dan *F1-Score* membuktikan bahwa retensi seluruh fitur

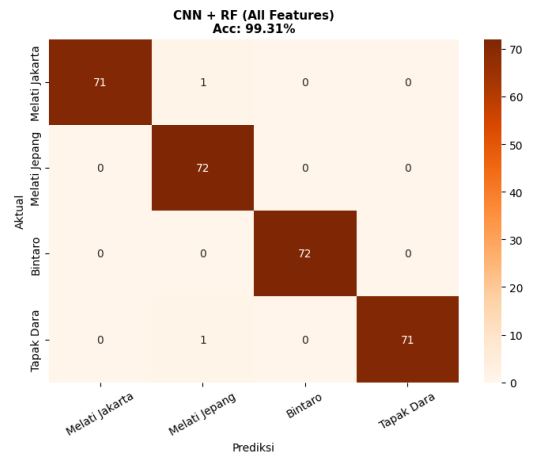
menghasilkan kinerja prediktif yang secara absolut lebih superior. Kernel Linear dan *Radial Basis Function* (RBF) mendominasi pada skenario "Semua Fitur" dengan capaian akurasi puncak masing-masing sebesar **98,3%** dan **97,9%**. Tingginya metrik pada kernel Linear secara spesifik mengonfirmasi bahwa vektor representasi fitur asli sudah memiliki tingkat separabilitas linear yang sangat optimal di ruang dimensi asalnya.



Gambar 3. 11 Confusion matrix SVM + VGG16

Pada confusion matrix, menunjukkan bahwa model berhasil mencapai tingkat pengenalan absolut (sensitivitas 100%) pada kelas observasi Melati Jakarta, Bintaro, dan Tapak Dara, dengan masing-masing memprediksi 72 sampel pengujian secara presisi sempurna. Penurunan akurasi secara eksklusif terisolasi pada kelas Melati Jepang, di mana dari 72 observasi aktual, 5 sampel mengalami galat klasifikasi (*misclassification*), di mana 3 teridentifikasi secara keliru sebagai Melati Jakarta dan 2 sebagai Tapak Dara, sehingga mereduksi metrik *recall* untuk kelas tersebut menjadi 93,05%.

3. 7. 6 Model Random Forest + MobileNetV2



Gambar 3. 12 Confusion Matrix RF + MobileNetV2

Berdasarkan *confusion matrix*, seluruh kesalahan klasifikasi terjadi pada kelas Melati Jakarta dan Tapak Dara yang masing-masing satu gambar salah diprediksi sebagai Melati Jepang. Kelas Melati Jepang dan Bintaro mencapai klasifikasi sempurna dengan 72 dari 72 gambar berhasil diprediksi dengan benar. Kesalahan pada Melati Jakarta dan Tapak Dara kemungkinan disebabkan oleh kemiripan visual tertentu dengan Melati Jepang dalam kondisi pencahayaan atau sudut pengambilan gambar yang spesifik.

BAB IV

PEMBAHASAN

4.1 Analisis Performa Model Terbaik

Serangkaian pengujian telah dilakukan, model *Deep Learning* dengan arsitektur VGG16 Fine-Tuning (pendekatan *end-to-end*) terbukti menghasilkan performa prediktif terbaik dengan pencapaian akurasi pada data uji sebesar 99,54%. Model ini mencatatkan nilai metrik yang nyaris sempurna, meliputi *Precision* (99,55%), *Recall* (99,54%), *F1-Score* (99,54%), dan *Matthews Correlation Coefficient* (MCC) sebesar 0,9939.

Keunggulan VGG16 dibandingkan model lainnya bersumber dari kedalaman arsitekturnya yang menggunakan tumpukan filter konvolusi 3×3 secara konsisten, sehingga memungkinkannya mengekstraksi representasi fitur yang sangat kaya dan berhierarki secara otomatis langsung dari piksel citra. Kualitas ekstraksi fitur tingkat tinggi ini terbukti mampu memecahkan ambiguitas visual pada kelas dengan kemiripan morfologi ekstrem. Sebagai contoh, pada metode *Random Forest* (ML), kelas *Jasminum* (Melati Jepang dan Melati Jakarta) menjadi sumber galat terbesar dengan *recall* Melati Jepang hanya mencapai 79%. Namun, setelah dilakukan penyesuaian bobot proksimal melalui *fine-tuning* pada lapisan *Conv5* VGG16, kesalahan prediksi pada kelas-kelas tersebut berhasil direduksi secara dramatis menjadi hanya 1 kesalahan dari 108 total citra uji.

4.2 Perbandingan Pendekatan *Machine Learning* dan *Deep Learning*

Penelitian ini memvalidasi keakuratan *deep learning* dibandingkan *machine learning* konvensional dalam menangani klasifikasi citra botani yang memiliki variasi intra-kelas tinggi. Model *Random Forest* dan SVM berkinerja sangat bergantung pada rekayasa fitur manual (*handcrafted features*) seperti *Histogram of Oriented Gradients* (HOG) dan statistik warna RGB. Meskipun SVM dengan optimasi parameter (HOG 8×8 , Orientasi 12) mampu mencapai akurasi maksimal 96,18% , dan *Random Forest* mencapai 88,19% , kedua algoritma ini memiliki keterbatasan komputasional dalam menangkap representasi semantik tingkat tinggi jika terjadi variasi pencahayaan atau sudut pandang yang kompleks.

Implementasi arsitektur CNN (*MobileNetV2* dan *VGG16*) secara mendasar membalikkan paradigma ekstraksi fitur manual menjadi pembelajaran fitur otomatis (*learned features*). Memanfaatkan bobot pra-latih ImageNet, VGG16 mampu mendongkrak akurasi hingga 11,35% lebih tinggi dibandingkan *Random Forest*. Lebih lanjut, pendekatan *Hybrid* yang mengintegrasikan ekstraktor fitur CNN dengan *classifier* konvensional juga terbukti sangat efektif. Model hibrida MobileNetV2 + Random Forest mencapai akurasi 99,31% , dan VGG16 + SVM (Kernel Linear) mencapai 98,26% , mengonfirmasi bahwa penggantian fitur HOG dengan fitur berdimensi tinggi dari CNN (512 dimensi pada GAP MobileNetV2 atau 4096 dimensi pada *fully connected* VGG16) sukses menghadirkan ruang vektor yang memiliki separabilitas linear yang jauh lebih tegas.

4.3 Kendala dalam penelitian

Selama proses pengembangan dan evaluasi model, terdapat beberapa kendala teknis dan analitis yang memengaruhi proses penelitian.

1. Perangkat yang dimiliki peneliti dengan spesifikasi yang masih kurang baik, memperlambat pelatihan dan pengujian model
2. Pada proses pelatihan fase kedua (*fine-tuning*) model VGG16, saat bobot lapisan konvolusi dibuka (*unfreeze*), model mengalami penurunan akurasi validasi yang drastis (dari 84,13% menjadi 73,56%) dan lonjakan *loss* di 5 *epoch* pertama. Fenomena penyesuaian bobot lintas-domain ini memaksa penggunaan *callback ReduceLROnPlateau* untuk menurunkan *learning rate* secara presisi agar model dapat kembali konvergen secara optimal di *epoch* selanjutnya.
3. Meskipun model berkinerja tinggi, *confusion matrix* lintas model secara konsisten menunjukkan tumpang tindih prediksi antara Melati Jakarta (*Jasminum sambac*) dan Melati Jepang (*Pseuderanthemum reticulatum*). Kemiripan struktur mahkota putih berukuran kecil pada kedua spesies tersebut menjadi batas tantangan klasifikasi yang paling sulit diisolasi oleh algoritma.

4.4 Kelemahan Model

Meskipun arsitektur VGG16 Fine-Tuning menghasilkan metrik akurasi tertinggi, model ini memiliki kelemahan fundamental dari segi efisiensi komputasi. VGG16 adalah arsitektur *high-capacity* yang sangat masif, memuat puluhan juta parameter (*trainable parameter* fase *fine-tuning* menyentuh 7,8 juta parameter). Hal ini mengakibatkan lambatnya proses pelatihan (memaksa *batch size* dibatasi hanya 16 sampel agar tidak terjadi *Out of Memory* pada GPU) dan waktu inferensi (*prediction latency*) yang lebih tinggi. Di sisi lain, model klasifikasi berbasis ML (*Random Forest* dan SVM konvensional) sangat bergantung pada optimasi ukuran dan orientasi bin *Histogram of Oriented Gradients* (HOG) melalui *trial and error* (*GridSearch*) yang menyita waktu, namun tetap gagal mendekati metrik akurasi *Deep Learning*.

4.5 Peluang Pengembangan ke Depan

Pencapaian penelitian ini membuka sejumlah potensi pengembangan riset dan aplikasi praktis di masa mendatang.

1. Implementasi Model Ringan (*Lightweight Models*) dengan *Attention Mechanism*

Riset selanjutnya dapat mengeksplorasi arsitektur CNN yang lebih ringan namun tetap *robust*, seperti modifikasi *ShuffleNet V2* atau *MobileNetV3* yang dipadukan dengan mekanisme *attention*. Langkah ini bertujuan mempertahankan akurasi 99% sembari mereduksi beban pemrosesan (*computational cost*) hingga ke pecahan detik.

2. Perluasan Dataset dan Kerangka Klasifikasi Ganda (*Dual-Output*)

Dataset dapat diperluas dengan mengakomodasi kondisi alam liar (*wild environment*), pencahayaan tidak ideal, dan penambahan jenis spesies endemik Indonesia lainnya. Selain itu, arsitektur *Deep Learning* ke depan dapat direkayasa untuk memprediksi tidak hanya kelas taksonomi bunga, tetapi juga tahap fisiologis (*bloom stage* seperti kuncup, setengah mekar, mekar penuh, dan layu) secara serentak (*simultaneous dual-output classification*).

3. Hilirisasi Produk Berbasis Kecerdasan Buatan

Model berakurasi 99,54% yang dihasilkan dalam penelitian ini sangat siap untuk diintegrasikan ke dalam sistem *backend* dari aplikasi ponsel pintar cerdas atau berbasis *web*. Sistem otomatis ini dapat dimanfaatkan secara langsung oleh penyuluh pertanian, industri florikultura komersial, maupun masyarakat awam untuk identifikasi taksonomi botani secara instan.

BAB V

Demo Program

5.1 Spesifikasi Perangkat Lunak dan Pustaka (Libraries)

Implementasi sistem komputasi untuk klasifikasi jenis bunga lokal dibangun menggunakan bahasa pemrograman Python. Seluruh dependensi peranti lunak dieksekusi di dalam lingkungan virtual (*virtual environment*) untuk menjamin isolasi pustaka dan reproduktivitas sistem. Spesifikasi pustaka utama beserta ekosistem pendukung yang digunakan dalam membangun demo program ini adalah sebagai berikut.

- Bahasa Pemrograman: Python versi 3.12.
- Framework Deep Learning menggunakan TensorFlow / Keras versi 2.15.
- Library Machine Learning menggunakan Scikit-Learn versi 1.4.
- Pemrosesan Citra Digital menggunakan OpenCV versi 4.9 dan Pillow (PIL) versi 10.2 (digunakan untuk operasi I/O citra, konversi ruang warna ke RGB/Grayscale, serta proses *resizing* matriks piksel).

5.2 Cara Menjalankan Program

Bagian ini menjelaskan langkah-langkah lengkap untuk mengunduh, menyiapkan lingkungan, dan menjalankan seluruh notebook klasifikasi gambar bunga dari repositori GitHub proyek ini secara lokal.

1. Persiapan Awal

Langkah pertama adalah mengunduh seluruh kode sumber proyek dari repositori GitHub. Pastikan Git telah terpasang pada sistem sebelum menjalankan perintah berikut. Buka *terminal* (Linux/macOS) atau *Command Prompt* / *PowerShell* (Windows), lalu jalankan perintah berikut.

```
git clone
```

```
https://github.com/rizkiiirr/FlowerClassificationusingDeepLearning.git cd FlowerClassificationusingDeepLearning
```

Apabila Git belum terpasang, repositori juga dapat diunduh secara manual melalui tombol Code → Download ZIP pada halaman GitHub, lalu ekstrak ke direktori yang diinginkan.

2. Persiapan *Virtual Environment*

Sangat disarankan untuk membuat *virtual environment* terpisah agar seluruh dependensi pustaka proyek tidak bertabrakan dengan instalasi Python global pada sistem. Jalankan perintah berikut sesuai sistem operasi yang digunakan.

```
python -m venv venv
venv\Scripts\activate
```

Setelah berhasil diaktifkan, nama (venv) akan muncul di awal baris *terminal* sebagai tanda bahwa lingkungan virtual sedang aktif.

3. Instalasi Dependensi

Seluruh pustaka yang dibutuhkan dapat dipasang sekaligus menggunakan file `requirements.txt` yang telah disertakan dalam repositori. Jalankan perintah berikut di dalam direktori proyek dengan *virtual environment* yang sudah aktif.

```
pip install -r requirements.txt
```

Proses instalasi memerlukan koneksi internet dan dapat memakan waktu beberapa menit tergantung kecepatan jaringan. Pastikan versi Python yang digunakan adalah Python 3.12 sesuai spesifikasi proyek.

4. Menyiapkan Dataset

Dataset gambar bunga yang digunakan dalam proyek ini tidak disertakan langsung di dalam repositori GitHub karena ukurannya yang besar. Dataset harus disiapkan secara terpisah dan ditempatkan di dalam direktori proyek dengan struktur folder sebagai berikut.

```
FlowerClassificationusingDeepLearning/
|
|— dataset/
|   |— melati_jakarta/      ← 360 gambar Melati
|   Jakarta
|   |— melati_jepang/      ← 360 gambar Melati
|   Jepang
|   |— bintaro/            ← 360 gambar Bintaro
|   |— tapak_dara/         ← 360 gambar Tapak Dara
|
|— Model_RF_HOG.ipynb
|— Model_SVM_HOG.ipynb
|— Model_MobileNetV2.ipynb
```

```

├── Model_VGG16.ipynb
├── Model_VGG16_SVM.ipynb
├── Model_MobileNetV2_RF.ipynb
└── requirements.txt

```

Nama subfolder di dalam direktori dataset/ harus ditulis persis seperti yang tercantum di atas karena seluruh notebook membaca nama folder tersebut sebagai label kelas secara otomatis. Link dataset tersedia pada Lampiran 2 laporan ini.

5. Menjalankan Notebook

Seluruh model diimplementasikan dalam format Jupyter Notebook (.ipynb). Untuk membuka notebook, jalankan perintah berikut di *terminal* dari direktori proyek. Apabila menggunakan VS Code, notebook dapat dibuka langsung dengan ekstensi Jupyter yang telah terpasang tanpa perintah tambahan. Apabila menggunakan Google Colab, unggah file .ipynb ke Colab dan pastikan dataset diunggah ke Google Drive, kemudian sesuaikan variabel DATASET_PATH pada sel konfigurasi.

6. Urutan Menjalankan Notebook

Proyek ini terdiri dari enam notebook yang masing-masing berdiri sendiri (*independent*) dan dapat dijalankan secara terpisah, sehingga urutan eksekusi yang direkomendasikan adalah Model_RF_HOG.ipynb, Model_SVM_HOG.ipynb, Model_MobileNetV2.ipynb, Model_VGG16.ipynb, Model_VGG16_SVM.ipynb VGG16 + SVM, dan Model_MobileNetV2_RF.ipynb

5.3 Uji Coba Pengujian

```

def predict_image(image_path, model, scaler, le, img_size=(64, 64)):

    # Load & preprocess
    img = Image.open(image_path).convert('RGB').resize(img_size)
    img_array = np.array(img)
    img_gray = color.rgb2gray(img_array)

    hog_feats = hog(img_gray, orientations=9,

```

```

        pixels_per_cell=(8, 8), cells_per_block=(2, 2),
        visualize=False)

color_feats = []
for ch in range(3):
    color_feats.append(img_array[:, :, ch].mean() / 255.0)
    color_feats.append(img_array[:, :, ch].std() / 255.0)

features = np.concatenate([hog_feats, color_feats]).reshape(1, -1)
features_scaled = scaler.transform(features)

# Prediksi
pred_idx = model.predict(features_scaled)[0]
pred_proba = model.predict_proba(features_scaled)[0]
pred_label = le.inverse_transform([pred_idx])[0]

# Tampilkan hasil
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].imshow(img)
axes[0].set_title(f'Gambar Input', fontsize=11)
axes[0].axis('off')

labels_display = [c.replace('_', ' ').title() for c in le.classes_]
bars = axes[1].barh(labels_display, pred_proba * 100,
                    color=[ '#E91E63' if c == pred_label else 'steelblue'
                            for c in le.classes_ ],
                    edgecolor='black', alpha=0.85)
axes[1].set_xlabel('Probabilitas (%)')
axes[1].set_title(
    f'Prediksi: {pred_label.replace("_", " ").title()}\n'
    f'(Confidence: {max(pred_proba)*100:.1f}%)',
    fontsize=11, fontweight='bold'

```

```

)
axes[1].set_xlim(0, 110)
for bar, val in zip(bars, pred_proba):
    axes[1].text(bar.get_width() + 1, bar.get_y() + bar.get_height()/2,
                  f'{val*100:.1f}%', va='center')

plt.tight_layout()
plt.savefig('08_demo_prediksi.png', dpi=150, bbox_inches='tight')
plt.show()

return {'kelas': pred_label, 'probabilitas': dict(zip(le.classes_, pred_proba))}

```

#	—	Contoh penggunaan
---	---	-------------------

```

# Ganti path di bawah ini dengan gambar yang ingin diprediksi
CONTOH_GAMBAR = os.path.join(DATASET_DIR, 'Tapak_Dara',
'Tapak_Dara_38_original.jpg') # Fixed path
hasil = predict_image(CONTOH_GAMBAR, rf_model, scaler, le)
print(hasil)

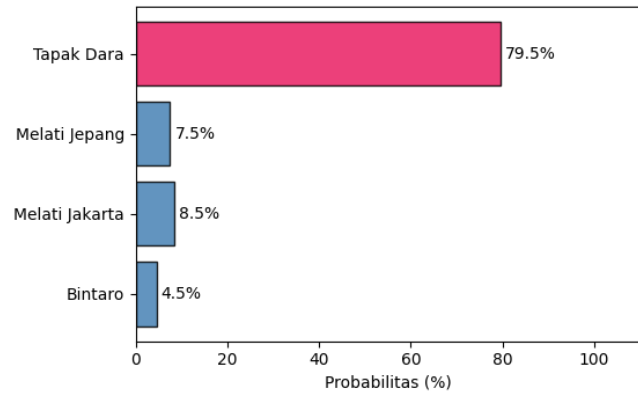
print("✅ Fungsi predict_image siap digunakan.")
print("  Hapus komentar (#) pada baris CONTOH_GAMBAR di atas untuk
mencoba.")

```

Gambar Input



**Prediksi: Tapak Dara
(Confidence: 79.5%)**



BAB VI

Kesimpulan

Berdasarkan seluruh tahapan eksperimen, analisis data, dan evaluasi komparatif yang telah dilakukan dalam proyek penelitian ini, dapat disimpulkan bahwa, penelitian ini berhasil mengatasi kendala ambiguitas visual dan keterbatasan identifikasi taksonomi manual pada empat jenis bunga lokal berwarna putih yang memiliki kemiripan morfologi mahkota tingkat tinggi, yaitu Melati Jakarta (*Jasminum sambac*), Melati Jepang (*Pseuderanthemum reticulatum*), Bintaro (*Cerbera manghas*), and Tapak Dara (*Catharanthus roseus*). Sistem otomatisasi yang dibangun terbukti mampu memisahkan batasan fitur visual objek secara objektif dan terukur. Kemudian, eksperimen diselesaikan dengan memetakan dan menguji keandalan dua pendekatan utama, yaitu metode *Machine Learning* konvensional sebagai *baseline* Random Forest serta SVM berbasis deskriptor geometris HOG dan metode *Deep Learning* berbasis *model-based fine-tuning* MobileNetV2, VGG1, serta arsitektur hibrida integratif VGG16+SVM dan MobileNetV2+RF. Hasil evaluasi empiris membuktikan secara konsisten bahwa pendekatan *Deep Learning* dan arsitektur hibrida memiliki performa yang jauh lebih superior dibandingkan *Machine Learning* klasik berbasis fitur manual (*handcrafted features*). Metode konvensional seperti Random Forest tunggal mencatatkan performa terendah dengan akurasi data uji sebesar 88,19% akibat ketidakmampuan algoritma mempartisi data citra mentah pada genus dengan kekerabatan dekat. Sebaliknya, pemanfaatan jaringan saraf konvolusi mampu mengekstrak representasi fitur hierarkis tersembunyi yang jauh lebih diskriminatif. Adapun arsitektur VGG16 Fine-Tuning merupakan model dengan performa prediktif terbaik di dalam penelitian ini. Model ini mencapai tingkat akurasi global tertinggi sebesar 99,54%, dengan nilai *F1-Score* seimbang di angka 99,54%, serta indeks korelasi metrik Matthews (MCC) yang menyentuh 0,9939. Model terbaik ini sukses mereduksi galat klasifikasi silang (*cross-classification*) pada subkelas *Jasminum* secara drastis hingga hanya menyisakan satu kesalahan interpretasi di sepanjang ruang pengujian data uji.

DAFTAR PUSTAKA

- [1] G. W. Intyanto, “Klasifikasi Citra Bunga dengan Menggunakan Deep Learning: CNN (Convolution Neural Network),” pp. 80–83.
- [2] B. Lee, “Classification of Germination Images of Pear Pollen Using Random Forest and Convolution Neural Network Models,” pp. 45993–45999, 2021, doi: 10.1109/ACCESS.2021.3067677.
- [3] E. Xi, “Image Classification and Recognition Based on Deep Learning and Random Forest Algorithm,” vol. 2022, 2022.
- [4] M. Cibuk, U. Budak, Y. Guo, M. C. Ince, and A. Sengur, “Efficient deep features selections and classification for flower species recognition,” vol. 137, pp. 7–13, 2019, doi: 10.1016/j.measurement.2019.01.041.
- [5] Y. Wu, Y. He, and Y. Wang, “Multi-Class Weed Recognition Using Hybrid CNN-SVM Classifier”.
- [6] Y. Fei, Z. Li, T. Zhu, and C. Ni, “A Lightweight Attention-Based Convolutional Neural Networks for Fresh-Cut Flower Classification,” *IEEE Access*, vol. 11, no. January, pp. 17283–17293, 2023, doi: 10.1109/ACCESS.2023.3244386.
- [7] S. Agrawal, R. Datta, A. Rudra, and C. Angione, “Comparative analysis of CNN-based approaches for flower classification,” *Smart Agric. Technol.*, vol. 14, no. November 2025, p. 102213, 2026, doi: 10.1016/j.atech.2026.102213.
- [8] S. Gupta and A. K. Tripathi, “Flora-NET : Integrating dual coordinate attention with adaptive kernel based convolution network for medicinal flower identification,” *Comput. Electron. Agric.*, vol. 230, no. December 2024, p. 109834, 2025, doi: 10.1016/j.compag.2024.109834.
- [9] I. Hossen, M. Awrangjeb, S. Pan, and A. Al, “Transfer learning in agriculture : a review,” no. December 2024, 2025.
- [10] M. Koklu, M. F. Unlersen, I. Ali, M. F. Aslan, and K. Sabanci, “A CNN-SVM study based on selected deep features for grapevine leaves classification (a) Ak (c) Büzgülü,” *Measurement*, vol. 188, no. November 2021, p. 110425, 2022, doi: 10.1016/j.measurement.2021.110425.
- [11] K. K. Gadiraju, “Via Deep Transfer Learning,” *IEEE J. Sel. Top. Appl. Earth Obs. Remote Sens.*, vol. 16, no. May, pp. 4699–4712, 2023, doi: 10.1109/JSTARS.2023.3270141.

Lampiran 1 Link Github

<https://github.com/rizkiiir/FlowerClassificationusingDeepLearning.git>

Lampiran 2 Link Dataset

<https://drive.google.com/drive/folders/1AcpKSbAGTPII0ihDS6Xer1WHGBVdgCFa?usp=sharing>

Screenshot Program dan Potongan Kode Penting.

```
def load_and_preprocess(dataset_dir, class_names, img_size):
    """
    Load gambar, resize, dan ekstrak fitur HOG.
    Mengembalikan array fitur X dan label y.
    """
    X, y = [], []
    for cls in class_names:
        cls_path = os.path.join(dataset_dir, cls)
        images = [f for f in os.listdir(cls_path)
                   if f.lower().endswith(('.jpg', '.jpeg', '.png'))]
        print(f" Memproses [{cls}] ... {len(images)} gambar")
        for img_file in tqdm(images, desc=f" {cls}", leave=False):
            try:
                img = Image.open(os.path.join(cls_path, img_file)).convert('RGB')
                img = img.resize(img_size)
                img_array = np.array(img)

                # Konversi ke grayscale untuk HOG
                img_gray = color.rgb2gray(img_array)

                # Ekstraksi fitur HOG
                hog_features = hog(
                    img_gray,
                    orientations=9,
                    pixels_per_cell=(8, 8),
                    cells_per_block=(2, 2),
                    visualize=False
                )

                # Fitur warna (mean & std tiap channel RGB)
                color_features = []
                for ch in range(3):
                    color_features.append(img_array[:, :, ch].mean() / 255.0)
                    color_features.append(img_array[:, :, ch].std() / 255.0)

                # Gabung HOG + warna
                combined = np.concatenate([hog_features, color_features])
                X.append(combined)
                y.append(cls)
            except Exception as e:
                print(f" ⚠️ Gagal load {img_file}: {e}")
```

6. Pelatihan Model Random Forest

```
▶ # — Inisialisasi dan training model —————
rf_model = RandomForestClassifier(
    n_estimators=200,      # Jumlah pohon keputusan
    max_depth=None,       # Kedalaman pohon tidak dibatasi
    min_samples_split=2,  # Minimum sampel untuk split node
    min_samples_leaf=1,   # Minimum sampel di leaf node
    max_features='sqrt',  # Fitur yang dipertimbangkan tiap split
    class_weight='balanced', # Menangani ketidakseimbangan kelas
    random_state=RANDOM_SEED,
    n_jobs=-1             # Gunakan semua core CPU
)

print(" ⏱ Melatih model Random Forest ...")
rf_model.fit(X_train_scaled, y_train)
print("✅ Training selesai!")

# Evaluasi pada data validasi
y_val_pred = rf_model.predict(X_val_scaled)
val_accuracy = accuracy_score(y_val, y_val_pred)
print(f" 📊 Akurasi Validasi : {val_accuracy*100:.2f}%")
```

7. Cross-Validation (K-Fold)

```
# — 5-Fold Stratified Cross-Validation —————
# Menggunakan train+val untuk cross-validation
X_trainval_scaled = scaler.transform(X_trainval)

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_SEED)
cv_scores = cross_val_score(
    rf_model, X_trainval_scaled, y_trainval,
    cv=skf, scoring='accuracy', n_jobs=-1
)

print("\n 📊 Hasil Cross-Validation (5-Fold):")
for i, score in enumerate(cv_scores, 1):
    print(f"    Fold {i} : {score*100:.2f}%")
print(f"    {'-'*25}")
print(f"    Mean   : {cv_scores.mean()*100:.2f}%")
print(f"    Std    : {cv_scores.std()*100:.2f}%")
```

8. Evaluasi Model pada Data Testing

```
# — Prediksi pada data test —————
y_test_pred = rf_model.predict(X_test_scaled)

# Hitung metrik evaluasi
acc = accuracy_score(y_test, y_test_pred)
prec = precision_score(y_test, y_test_pred, average='weighted')
rec = recall_score(y_test, y_test_pred, average='weighted')
f1 = f1_score(y_test, y_test_pred, average='weighted')

print("=" * 45)
print("    HASIL EVALUASI MODEL - RANDOM FOREST")
print("=" * 45)
print(f"    Accuracy   : {acc*100:.2f}%")
print(f"    Precision   : {prec*100:.2f}%")
print(f"    Recall     : {rec*100:.2f}%")
print(f"    F1-Score   : {f1*100:.2f}%")
print("=" * 45)
```